M362 Unit 11
UNDERGRADUATE COMPUTING

Developing concurrent distributed systems

Beyond Java: Heterogeneous distributed systems

Unit 11

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

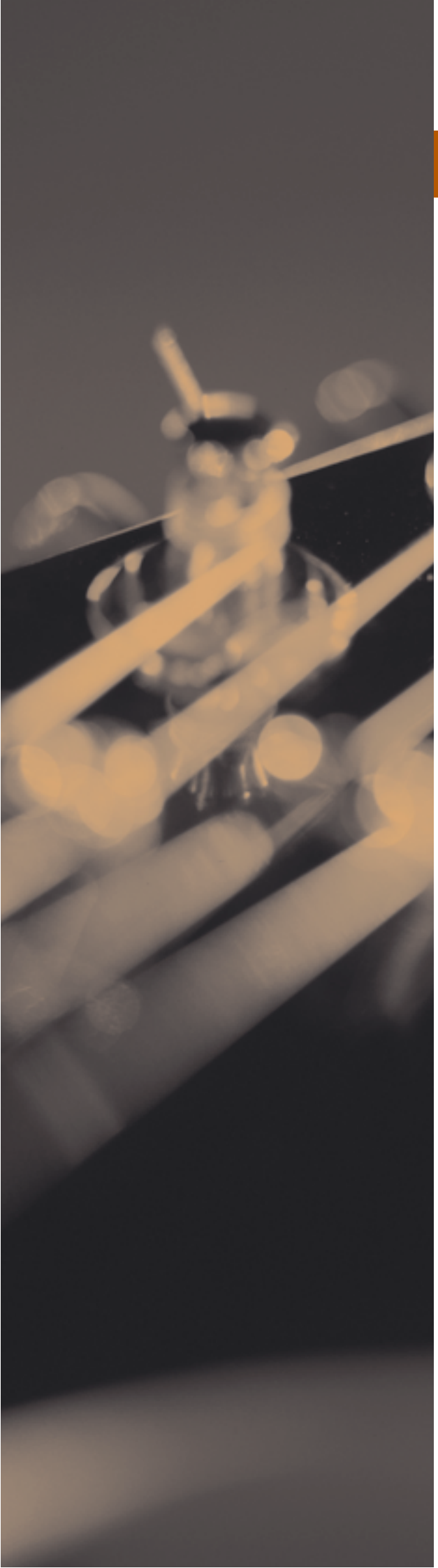
Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1599 6

1.1



CONTENTS

M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1 Introduction

This unit is about techniques for supporting **heterogeneous systems**, that is, systems implemented using more than one kind of computer hardware, operating system, programming language or communication protocol. These techniques also apply to distributed systems, which are also often heterogeneous, because they may have been developed in different locations at different times and for different purposes.

Early computer platforms supported only their native operating systems and programming languages and therefore software could not be moved from one platform to another without considerable effort and rewriting. To port software, programmers would have to implement low-level communication facilities and to cater for differences in representation of data, such as how strings or integers were stored by each system. However, these obstacles had to be overcome in order to reap the benefits of combining the power of different software packages.

Today, the need for communication between a variety of computer systems remains, for a number of reasons:

- continued use of legacy systems such as older databases;
- increased expectation of easy access to distributed systems, regardless of location;
- use of mobile devices such as phones and PDAs, and the desire to be able to access information using these devices via the internet;
- 'convergence' of technologies, that is, shared capabilities of devices – for example, mobile phones now frequently come equipped with cameras, mp3 players and radios, and so handle a variety of media and give the user the opportunity for sharing these media with related devices or uploading them via networks to other systems.

Multimedia fridge

An unusual example of convergence is given by the company LG's 'digital multimedia fridge' that has been available for a few years. The following text is taken from their online advertisement.

> Watch TV, listen to music or surf the internet using this titanium finish, state-of-the-art fridge freezer. It's the ultimate in kitchen technology with a built-in MP3 player for downloading and playing music from the internet, e-mail and video mail using a built-in camera and microphone. It even has full internet access so you can re-stock the refrigerator on-line or check on the latest news and weather – all without leaving the kitchen. And it's great for storing food too. ... What's more ... it diagnoses minor faults on-screen and has a contents page for entering and monitoring food content and expiry dates.
>
> (LG Electronics, 2002)

Digital multimedia fridge freezer.

This kind of functionality requires **interoperability**, that is, a facility for easy communication, and we will discuss ways by which this can be achieved through the use of **middleware**, which is software that masks the heterogeneity of underlying systems.

Middleware assists us in creating 'pluggable' software components, that is, components that can be connected together easily and can communicate transparently with each other.

1.1 The aims of this unit

In this unit we will look at examples of middleware in:

- CORBA (Common Object Request Broker Architecture), which is a specification enabling the creation of software components that communicate using the ORB (object request broker) paradigm;
- SOAP, a messaging protocol supporting heterogeneous communication.

We will also discuss:

- XML, a flexible language for describing data;
- web services, which are web-based software components – SOAP is one way of supporting these services;
- the Microsoft .NET (pronounced 'dot net') and Java EE platforms, giving you an overview of how they solve the problems of heterogeneous communication and of how these platforms compare with each other.

This unit involves reading together with some exercises and practical activities. There are also suggestions for further reading on the course website.

2 Middleware

Middleware is the general term for the software that is located between other software components in a system. Its chief aim is to mask the heterogeneity between the other parts of the system, so that they become interoperable. Throughout this course you have already come across a number of different examples of middleware. In this section we provide an overview of some general techniques to facilitate the writing of middleware using:

- software components presenting a standardised interface;
- intermediate layers that facilitate communication between heterogeneous systems;
- a registry to provide a way of discovering services offered by heterogeneous systems.

2.1 Components

We are surrounded by technology that relies on the existence of standardised interfaces. Chaos would ensue if electric plugs, light bulbs, fuses, transistors or computer components such as hard drives did not conform to certain expectations. (Admittedly, there are many different interfaces for some of these items!)

For many years the idea of extending the advantage of standard interfaces to **software components** has been discussed. This is desirable because components, by definition, are composable; that is, they can easily be combined to create a higher level of functionality than they have individually. For example, you may have a spreadsheet or a calendar component that can be embedded in a web page to provide extra functionality to the web page. With standardised interfaces, components can also more easily make use of middleware to extend their functionality and communicate with other components.

In order to be composable, components must have certain characteristics. As discussed in *Unit 5*, we may refer to a **component model** as a set of standards for the implementation, documentation and deployment of components. If software conforms to such a model, it becomes composable and hence reusable.

Desirable properties for a component include the following.

- A well-defined interface, that is, an API that encapsulates and provides access to data.
- A means of revealing information about itself, that is, 'introspection', or self-knowledge. For example, this may include revealing the component API, a description of the properties of the component, or a description of the kinds of data that may be available from it.
- A means of initiating actions, or indicating that a certain event (such as a user clicking on a button) has taken place.
- Persistence, or the ability to store and retrieve the state of an object (in Java, provided by `Serializable` classes and the Java Persistence API).
- A means of instantiating the component (an object-oriented property).
- Portability, which may also be supported by middleware.

There are several major competing models for components, however. Some examples are the following.

- The Microsoft Windows Component Object Model (COM), which later led to the Active X control, and was also extended to the Distributed Component Object Model (DCOM), which is Microsoft's implementation of RPC (remote procedure call) for use in networks. This has now been somewhat superseded in the .NET platform.
- The server-side component model provided by Enterprise JavaBeans (EJB), discussed in *Unit 7*.
- JavaBeans, discussed in *Unit 8*. JavaBeans must conform to a number of conventions; for example, provision of a zero-argument constructor, naming conventions for setter and getter methods, and serialisability. Introspection in Java is provided by the set of classes referred to as the Reflection API – these classes also provide the ability to alter objects' state and behaviour at runtime. The Reflection API supports visual editing of JavaBeans, as a bean's properties can be determined by inspecting its class directly.

Project Matisse.

NetBeans' GUI builder, Matisse, is an example of a visual editor.

Because there are several incompatible component models, software components may not be interoperable. In the following subsections we discuss some general techniques for achieving interoperability between components and how this leads us to the idea of middleware.

2.2 Layers of indirection

Many problems in computing may be solved through communication between systems or their parts. Two issues with this problem-solving approach are of immediate concern.

- Communication may not be supported, because there is no shared language.
- Changes to either system after communication has been established may mean that communication is no longer possible, or will lead to incorrect results.

However, it is possible to attack both of these issues with the same approach, namely, using an intermediary to communicate.

A similar principle has been stated many times, leading to a famous dictum:

> Every problem in computer science can be solved by adding another level of indirection.
>
> (Attributed to either Microsoft's Butler Lampson or Cambridge University's David Wheeler)

By **indirection**, we mean an indirect interaction, or an interaction via an intermediary that can communicate with both sides.

While this principle is not intended to be taken too literally, it has a strong element of truth in it: as you saw in *Unit 5*, the term middleware encompasses a number of different software technologies that use indirection to solve communication problems. The key role of middleware is to act as an intermediate layer, so that components do not need to be rewritten for various operating systems or to provide a variety of modes of communication beyond the ability to communicate with the middleware. In this way, middleware hides heterogeneity in underlying platforms, helps to provide location transparency and also helps to insulate components from changes to other components.

Rather like a JVM (Java Virtual Machine), middleware needs to be written to cope with different underlying platforms, but presents a standardised interface to the programmer creating an application and so facilitates communication between

applications. These features are illustrated in Figure 1, in which, for example, application A could be a client application written in one programming language and application B a server written in a different programming language and running on a different operating system.

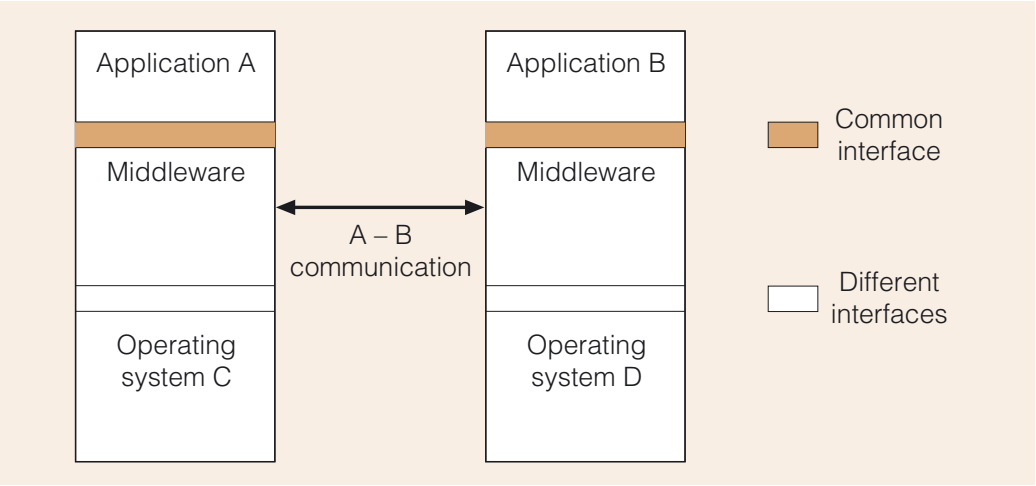


Figure 1 Middleware as a layer between operating systems and applications

Direct communication between applications A and B would be difficult to achieve, but with the use of a layer of middleware for each operating system, this becomes much easier to manage.

Middleware provides a higher level of abstraction than an underlying system, which makes for easier programming. Thus, middleware provides a building block for the construction of distributed software components.

Part of the communication task is the process of marshalling and unmarshalling data, using external data representations compatible with both platforms involved. For example, a middleware application may use ASCII or binary in various forms, serialised (flattened) data in the case of Java RMI (Remote Method Invocation), or (as you will see) the Common Data Representation (CDR) in the case of CORBA, to represent data.

In simpler terms, if communicating systems speak a common language (Esperanto, if you like) it does not matter if their native languages are different, provided that the intermediate language has sufficient expressiveness. Neither does it matter if the individual systems change, provided that they can continue to speak Esperanto.

Esperanto is a spoken language invented in the late nineteenth century, intended to foster better understanding between speakers of different languages.

Other notable examples of common forms of communication in distributed computing include TCP/IP for internet communication, HTTP for web page requests and HTML for web page content. Such standards form a foundation on which much else is built, freeing middleware programmers from consideration of many low-level details.

Several early middleware implementations (for example, the Distributed Computing Environment) were based on a non-object-oriented, client–server approach in which the middleware listened for requests from a client and translated them into a form acceptable to a server. More recently, object-oriented middleware such as RMI and CORBA has come to the fore.

Exercise 1

What other examples of indirection can you think of that we have encountered in this course already?

Discussion .....

Some examples we thought of are:

- the Java Virtual Machine;
- accessing data using setters and getters instead of directly;
- RPC hiding of details of socket communication from programmers, relieving them of dealing with some of the low-level details of distributed communications;
- proxies in RMI;
- message-oriented middleware;
- the ORB.

You may have come up with other examples.

2.3 Aspects of middleware use

We now consider some aspects of the efficiency, scalability and robustness of middleware that are important considerations when deciding what approach to employ in a distributed system.

Loose versus tight coupling

Middleware and indirection in general provide **loose coupling**. This means that when the communicating parties change internally, communication between them need not change because the interface provided by the middleware has not changed, and this supports greater reusability and robustness.

However, it has also been noted that the indiscriminate use of intermediate layers may result in an avoidable performance penalty, and so we also have the humorous riposte to our first dictum:

> Every performance problem can be solved by removing a level of indirection.
>
> (Author unknown, attributed by some to M. Haertel)

In other words, **tight coupling** (direct communication without an intermediary) can have efficiency benefits. Occasionally it may be preferable to bypass an intermediate layer, to avoid the overhead of translating between data representations and thereby increase efficiency. However, tight coupling also means that the ability to communicate can be affected by changes to either of the communicating parties.

It is also possible to provide for loose coupling and hence greater scalability through replication of services. For example, in a client–server system, there could be multiple servers and so the choice of server may be made at runtime. The client need not know the location of the server in advance; rather it may be selected from a list of possible servers by an intermediary. Such an intermediary could also make decisions based on how busy the servers are, or whether some servers are currently unavailable, without affecting the client.

Note also that replication of services leads to greater robustness through redundancy. This approach can be taken in a variety of systems, including CORBA and web services.

A developer will have to consider whether to apply loose or tight coupling in different parts of the system, depending on whether efficiency is considered more important than the ability to easily modify either of the communicating parties.

Synchronous versus asynchronous communication

In *Unit 5* you saw a number of paradigms for distributed object communication and you have seen that a general characteristic of any communication paradigm is that it may be synchronous or asynchronous, that is, it may be blocking or non-blocking.

Synchronisation implies tight coupling in terms of timing.

When planning use of a middleware, it is worth considering the effect that the choice of synchronous or asynchronous communication has on the system as a whole in terms of scalability and robustness. In general:

- Synchronous communications tend to greater robustness, because the calling side is provided with an opportunity to detect errors in the communication and recover from them. However, this approach is less scalable, because bottlenecks in performance can become more apparent.
- Asynchronous communication scales better, because the sender of a communication is not locked into waiting for a response. However, this approach tends to less robustness, as there is less scope for recovery when things go wrong.

For example, in *Unit 5* we considered the use of the Java `Socket` class, which creates a blocking socket. (Even if you create a thread to handle a client, that thread will be subject to blocking.) It is also possible to create non-blocking sockets (in Java this is done using the class `SocketChannel`) and so socket-based communication can be asynchronous or synchronous. Non-blocking sockets allow for asynchronous communication and this provides a highly scalable approach, while blocking sockets may result in multiple connected clients and threads, and so are less scaleable. On the other hand, blocking sockets provide for greater recoverability and robustness, as there is an opportunity to detect failure and recover from it.

SAQ 1

(a) What is a component?

(b) What is a component model?

(c) What are the chief advantages of using components and middleware?

ANSWER.....

(a) A component is a composable software element conforming to a component model.

(b) A component model is a set of standards for the implementation, documentation and deployment of components. For example, a component model may describe the nature of the interface provided by a component and how a component reveals information about itself.

(c) Components are composable, and support interoperability. Interoperability is also supported by middleware, which provides a layer of indirection and hence also looser coupling, which leads to greater scalability and more robustness (sometimes at the cost of some efficiency).

2.4 Services and discovery

As interoperability improves, it becomes possible to discover useful components or, more specifically, the **services** they provide. By services we mean the various kinds of work components can do for you, or information they can provide.

As an example, you may have a client that requires a printing service and does not require a particular printer or print server to be used. All the client may need to know is that some server provides a print method with an appropriate signature. The actual choice of server can be made at runtime, provided that there is a mechanism by which the client can look up the required information. This process can be made completely transparent to the client.

Unit 7 used the term 'directory service'.

A component can provide a **discovery service** to respond to client requests for information about available services, and enable a client to access them.

Creating a discovery service requires:

- a **registration service**, by which servers can record information in a repository of some sort, so that clients can look it up;
- a **lookup service**, by which clients can ask for information about registered services, choose between various services, or have a service chosen for them.

You have already seen these ideas in the RMI registry, which also provides a name lookup service to client applications. We also discussed the idea of a naming service in *Unit 7*, in the form of the DNS (Domain Name Service), which is essentially a lookup service. The use of a repository also supports replication as it can contain information about more than one service provider, and this can help to create location transparency.

Discovery services may employ a variety of parameters to decide which services they know of (if any) that are suitable for a client's needs. This is called the **scope of discovery**. For example, in the case of the client requiring a printing service, only colour printers within 200 metres of the client's mobile device may be considered to be in scope.

We will consider examples of discovery services in CORBA and web services later in this unit.

Jini network technology.

Jini

Jini is an example of a software technology supporting sharing of computing resources and the use of 'dynamic' networks, that is, networks that are continually adding or removing nodes and adapting to change. As such, Jini adds some scope for added reliability to services built from unreliable parts.

Jini allows executable code to be passed from one network node to another in the form of Java objects and has been used as the basis of some grid computing projects and in ad hoc networking (for example, telematics) amongst other applications. It can also be used to share web services and to evolve services without interrupting them.

2.5 Interface definition languages

The idea of an interface definition is familiar to us from Java interfaces, which describe how to invoke an object's methods. Interfaces support loose coupling; that is, systems need not be aware of implementation details of the services they want to access, only of the interface to those services.

There is always a danger of drift between services implied by an interface and those expected by a client. Meta-information services can help to mitigate this.

Services may likewise be described in terms of their interfaces. The separation of the interface from the business logic supports component interoperability and is the basis of a **service-oriented architecture**, in which the main unit of communication is a message (as opposed to a method invocation). Messages are sent to service providers using a supplied interface, which works as a kind of façade to provide access to the business logic behind it. Service-providing components manage their own data and security boundaries and so can be connected to each other in arbitrary configurations.

For the purposes of middleware, an implementation behind an interface may have been created in one of a variety of programming languages and so an interface needs to be provided in a platform- and language-independent way, which means allowing for different forms of communication supported by different languages. An **interface definition language** (IDL) provides this language independence.

An example of language differences occurs with their provided types. For example, not all languages have a long integer type. Even if two languages both have a long type, one may be six bytes in length and the other eight bytes. An IDL must specify a set of types that may be used for communication between supported languages. Thereafter only those types may be employed in communications. Likewise, the semantics of an operation invocation may vary from one language to another, and the IDL must cater for such differences.

We say that an IDL provides a **mapping** to the languages that it supports. Thus an IDL allows a language-independent description of message-passing behaviour. However, it is quite possible that a language does not map completely on to an IDL and so only some features of the language might be supported.

The ideas we have mentioned in this section – layers of indirection, components, service discovery and standards for describing services – provide a powerful set of tools for creating heterogeneous and distributed systems.

SAQ 2

(a) What is an IDL?

(b) Which is typically more robust: synchronous or asynchronous communication? Why?

(c) What is the relationship between middleware and the tightness of coupling between components?

ANSWER.....

(a) An IDL is a language for describing the interface of a component, such as, for example, a Java `interface` description, but it also provides mappings to various supported languages, catering for differences in semantics and data types between languages.

(b) Synchronous communication is typically more robust because it involves a 'handshake' between the two sides (one side must wait for the other before proceeding) and this provides an opportunity that might not be available in an asynchronous communication to recover from errors.

(c) Middleware promotes loose coupling in that components communicate via the middleware and need not use component-dependent means of message passing.

3 CORBA

In *Unit 5* we introduced the idea of the object request broker (ORB), in which one object communicates with another (local or remote) object via an ORB. An important advantage of this approach is that the ORB can act as an intermediary and this means that the communicating objects may be implemented in different programming languages.

In this section we discuss a specification of the ORB model known as CORBA, the **Common Object Request Broker Architecture**, which defines the facilities necessary for distributed objects to communicate with each other using this paradigm.

CORBA is particularly intended for use in large, distributed, heterogeneous systems, including ones created on different platforms and whose components are written in different languages. In particular, CORBA can enable legacy systems to be accessed in a transparent way and provides important distributed system services, such as support for naming of distributed objects, and for transactions and security.

Because of these advantages, other systems are often built on top of CORBA.

Sun Microsystems suggests that CORBA complements Java in that Java provides 'write once run everywhere' portability, while CORBA provides a distributed objects framework, services to support that framework and interoperability with other languages.

CORBA support is included in Java EE and core Java in the form of the Java IDL, which allows CORBA programmers to communicate with Java objects using standard CORBA techniques. Where communication from Java to heterogeneous systems is desired, CORBA can also be used to support RMI communications. In fact, CORBA is similar to RMI in many ways, with one of its main differences being its support for different implementation languages.

3.1 The Object Management Architecture

CORBA is a specification that is part of a larger model called the **Object Management Architecture** (**OMA**), produced by the **Object Management Group** (**OMG**), a non-profit consortium with many hundreds of corporations amongst its members. The architecture dates back to 1991 and is under continuing development.

UML, the Unified Modelling Language, is one of the OMG's specifications.

The OMG describe the Object Management Architecture as an

> open, vendor-independent specification for an architecture and infrastructure that computer applications use to work together over networks.
>
> (OMG, 2007)

The vision of the OMG is that objects should be built to work as components in software systems across distances and platforms, over networks. That is, objects should be able to communicate with each other no matter where, or on what type of machine, they are located. The implementations of CORBA **objects** need not be object oriented, provided that they are written in a language supported by the ORB employed.

To achieve this vision, the **OMG Object Model** provides a platform-independent way of specifying the externally visible characteristics of objects, and describes some of the main concepts required, including definitions of the following.

- CORBA types.
- CORBA object **operations** (which may also be referred to as services and are the equivalent of object methods). An operation may, for example, allow the querying of the state of a CORBA object or have it execute business logic.
- Object services, described using CORBA **interfaces** (which are similar to Java interfaces, but are written in an implementation-independent way).
- Operation semantics.
- Object attributes.
- Exceptions, which contain information about errors leading to abnormal termination of CORBA programs.

The ORB is central to the goal of distributed heterogeneous object communication, because it acts as a sort of communication bus and routes messages between parts of a CORBA-based system. Thus, the ORB is a middleware that deals with requests for **object services** from one CORBA object to another, dealing with requests to invoke operations on objects.

Strictness (or weakness) of specifications is a key issue with the OMA.

The CORBA specification itself describes the interface to the ORB, and there are other related specifications within the OMA describing interfaces to various other services. As services are described by interfaces, the implementations of the services themselves may vary. For example, they may be at various levels of quality of service. An event notification service may be 'fast and unreliable' or 'slow and reliable' while having the same generic interface. This allows a client to select an implementation according to need, using a consistent interface.

There are implementations of CORBA that operate using only a single language, such as Java or C++. Thus a CORBA implementation may be compliant without supporting heterogeneous communication.

Software implementing the CORBA specification is said to be **CORBA compliant**.

Figure 2 illustrates the OMA Reference Model, which defines the various component types in the architecture.

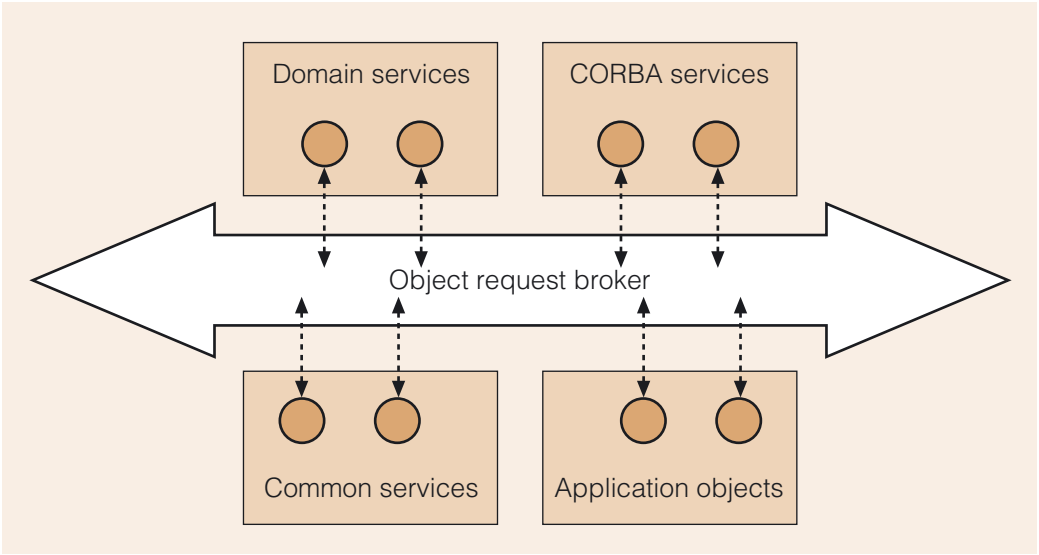


Figure 2 The Object Management Architecture Reference Model

CORBA objects can be grouped into several types.

Each object type has its own OMG specification.

- **Domain service** objects conform to CORBA specifications for systems within domains with special requirements, such as medical or financial systems, real-time systems, and platforms that have little memory or processing power (such as PDAs).
- **CORBA service** objects provide a number of standard services that are important for distributed systems, such as support for naming and transactions.
- **Common service** objects provide generic services that are useful across applications and domains, including support for printing, mobile agents and internationalisation.
- **Application objects** are user-defined objects. An application object could, for example, describe a shopping basket for an online shop. It would then be possible to create multiple instances of shopping baskets, one per customer.

A **CORBA server** is an object providing a service, while a **CORBA client** is an object requesting a service.

Server objects initialise a CORBA environment and instantiate **servants**, which are visible only to the server and represent the resource that performs an object's operation. A client therefore does not need to know the actions a server is taking behind the scenes in terms of how many servants have been created or where their resources are located.

The remaining parts of this section describe application objects, CORBA services and communications between parts of the system, but we will not be considering the domain or common services further.

SAQ 3

(a) What is the purpose of the OMG Object Model?

(b) What does CORBA stand for and what is its purpose?

(c) In what ways are the common services, CORBA services and domain services in the OMA Reference Model different from each other?

(d) In the OMG Object Model, what are operations used for?

ANSWER.....

(a) The OMG Object Model defines a standard platform – an implementation-independent way of specifying the externally visible characteristics of objects, such as their types, operations and interfaces. This means that it supports object invocation by clients on target objects no matter where in a distributed system, or on what type of machine, they are located.

(b) CORBA stands for Common Object Request Broker Architecture. It is a specification for a CORBA-compliant ORB, which routes communications between objects.

(c) The context served by these components differs.

Common services are of general use, such as printing and internationalisation.

CORBA services support the requirements of distribution, such as a naming service and a transaction service.

Domain services are of direct interest to end-users in particular domains, such as medical or financial applications.

(d) Operations are equivalent to methods; they enable the querying or changing of a CORBA object's state and the execution of business logic.

3.2 Describing CORBA objects

Before we describe the core of this architecture, which is the ORB, we consider how implementation independence is achieved in the description of object services.

In order to describe object services, we need to be able to name object types and describe their interfaces in a way that is independent of the object's implementation language. To achieve this, CORBA includes an IDL of the sort we discussed in Subsection 2.5.

The OMG IDL

The IDL defined by the OMG, the **OMG Interface Definition Language** (**OMG IDL**), is an ISO standard (ISO/IEC 14750) and is an object-oriented description language that includes standardised mappings to versions of both object-oriented and non-object-oriented languages, including C, C++, Java, Smalltalk, Cobol, Ada, LISP, PL/1 and Python, amongst others. The outputs of such mappings are predictable and will not vary from one run to another.

To represent data in a consistent way, there is a type set called the **Common Data Representation** (**CDR**) that includes:

- primitive types such as various precisions of integers, floating-point numbers, characters, a Boolean type and a fixed-point number type;
- container types such as arrays, variable and fixed-length strings and a sequence type;
- an enumerated type and user-defined types constructed from more complex record-holding types such as `struct` and `union` types;
- an `interface` type that is used to describe CORBA object behaviour;
- an `any` type, for when the type is arbitrary.

Structs and unions are similar to classes that have no methods.

The sizes of all types are specified to ensure interoperability between CORBA implementations. Language-specific types are converted into CDR during marshalling and back into native types during unmarshalling. This means that type safety is provided for the invocation of operations.

RMI is homogeneous and so there are no IDL mapping issues relating to varieties of different language types or their sizes. A homogeneous approach may also require less learning and provide a more rapid implementation.

The `interface` type

The most important type we will consider is the `interface`, which resembles an interface in Java in that it may contain CORBA operation signatures (similar to Java method signatures).

You may have guessed from the fact that there is a Java to IDL mapping, that the OMG IDL also supports the description of exceptions. Each operation may have a `raises` clause and there is an `exception` type, which is used to specify predefined or user-defined exceptions that operations may raise.

A CORBA interface allows for a richer description of object behaviour than a Java interface and may also define the following.

- The semantics of an operation invocation; for example, whether an invocation should be synchronous (blocking) or not.
- `attribute` names, which are a shorthand for variables that may have a setter operation and a getter operation. (A `readonly` attribute has no setter.) The required operation names are implied from the attribute names and modes, as illustrated in the example below.

- `constant` values.
- User-defined types, using `typedef` statements. These types may then be used wherever they are required and will be subject to type checking as with any other type.

Definitions such as interfaces and exceptions are collected together in an IDL module, which corresponds roughly to a package in Java and provides a way of scoping names.

For example, in an application for our Music Store, purchasing an instrument might involve invoking operations in a `checkout` interface. The following module might be provided to describe this interface.

```
module MusicStore
{
  exception BadPerc
  {
    string reason;
  };

  interface checkout
  {
    attribute float balance;
    readonly attribute string owner;

    float calculateTax(in float amount);
    float calculateDiscount(in float percent) raises(BadPerc);
  };
};
```

This definition produces a **CORBA object type** describing a CORBA `checkout` object and its operations.

Notice that the operation parameters have not only a type, but also a **mode** (`in`). An `in` parameter may be read from, but not written to. The other modes are `out` (may only be written to) and `inout` (which may be read from and written to).

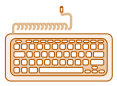
A module called `CORBA` is inherited by all interfaces and contains interfaces for communication between various parts of the system, as well as other special interfaces such as `Object`, which provides low-level, reference-handling operations for all CORBA objects.

Every ORB implementation will provide a set of IDL compilers for its supported languages. An **IDL compiler** is used to compile a module and will produce a **stub** for the client and a **skeleton** for the server side, compatible with the programming language used on each side, as well as supporting files that provide the 'plumbing' behind the scenes to communicate with an ORB.

Stubs and skeletons in CORBA are similar to proxies in RMI. An IDL stub issues requests on behalf of a client. A skeleton behaves as a proxy for the servant.

For example, the following Java interface can be generated from our example module using an IDL compiler for Java.

```
package MusicStore;
/**
 * MusicStore/checkoutOperations.java
 */
public interface checkoutOperations
{
  float balance();                          //getter
  void balance(float newBalance);           //setter
  String owner();                           //getter
  float calculateTax(float amount);
  float calculateDiscount(float percent) throws MusicStore.BadPerc;
}
```



Activity 11.1
CORBA 'Hello World'.

Notice that the compiler created both a setter and a getter for the `balance` attribute, but only a getter for the `owner` attribute.

The skeleton produced by the IDL compiler is an abstract class that provides code to deal with the requirements of the ORB communication. The servant in this case would simply be a standard Java class that extends the skeleton class and implements the required interface, in this case the `checkoutOperations` interface.

The IDL compiler also provides files on the client side, including a stub that implements the same interface and is used by the client side to communicate with an ORB.

We have shown an example in Java; however, more generally, a programmer must provide an implementation of the CORBA interface in a supported language, and instances of these CORBA objects can then be employed on any platform that the ORB supports.

A server class creates instances of servant objects and registers them with an **implementation repository**. In other words, information about using the server and servant objects becomes part of a database. For example, this repository will include information such as the host and port number to use to connect to a server object. An ORB can use the information in the repository to determine whether or not a suitable server is running, and the server can create a servant when needed, if necessary.

Dynamic interfaces

We have presented an interface as a static way of describing the services available; if the interface is changed, the client stub and the server skeleton need to be regenerated. Much of the time this is sufficient, but sometimes there may be a need to dynamically discover a server interface of a certain type, or a need to change server interfaces while a system is running.

To support dynamic creation and discovery of services, an ORB may provide an **interface repository**, which maintains type information about IDL files. This database can be used by a server at runtime to store information about interfaces it supports. A client can use this repository to dynamically locate an object type without the use of a stub and to determine how to construct a request to a servant of that type. In other words, the repository can provide a client with an object reference based only on its type, and the client can then query the operations the related servant object supports.

Further support for this dynamic capability is provided by the **Dynamic Invocation Interface** on the client side, which allows the client to act without a static stub. On the server side, the related component is the **Dynamic Skeleton Interface**, which allows a running server to dynamically create a new interface type. These components can be seen as providing generic stubs and skeletons on the client and server sides.

SAQ 4

(a) What is the OMG IDL and why does CORBA support it?

(b) What is a language mapping and what is it used for?

(c) What is the purpose of a CORBA interface?

(d) What is an interface repository and what is it used for?

ANSWER.....

(a) The OMG IDL is the interface definition language in the OMA for defining the interfaces of objects (i.e. the operations and the types of object) in a manner that is programming-language-independent. Interfaces are defined separately from object implementations.

CORBA supports it because it enables the description of services provided by objects written in different programming languages.

(b) For each programming language, it must be possible to create an IDL interface. A language mapping maps the data types found in the language to the CDR in CORBA. The OMG has standardised mappings for a variety of languages. This provides type safety for operation invocation.

(c) An interface describes a CORBA object type and the operations supported by that type of CORBA object. Interfaces also hide object language implementation details, so supporting implementation heterogeneity. Indirectly, interfaces also allow the specification of substitutability of one CORBA type for another and one operation for another.

(d) An interface repository holds the specifications of object interfaces written in the IDL. An interface repository allows the dynamic discovery of operations provided by objects, given a CORBA object type.

3.3 The ORB

An ORB's main responsibility is for routing requests between clients and servants. The functionality of the ORB is very similar to that provided by Java RMI and includes:

- locating servers able to provides services to other objects;
- preparing objects to receive client requests;
- communicating data between two objects taking part in a software interaction, including the return of results from a servant.

An ORB can be used to support heterogeneous communication in a local context as illustrated in Figure 3.

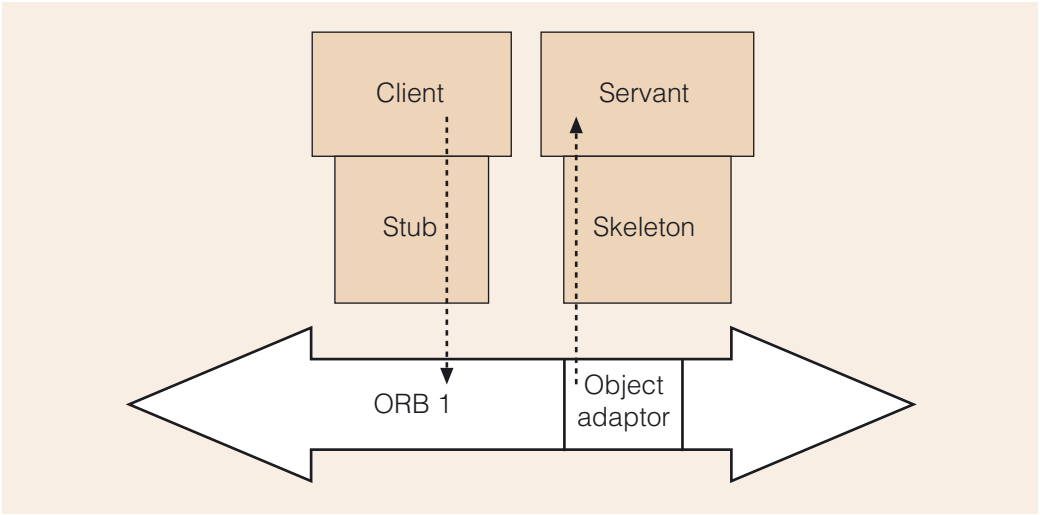


Figure 3 A local ORB operation (a call to a servant)

We have omitted the server from our figure, as the client communication is essentially with a servant object.

In the typical case, when a client object requires a service, it obtains a reference to a servant object by requesting that a naming service looks up the object associated with a name, which is called **resolving** the name. The client can then invoke an operation on the resolved reference, using a stub that communicates with an ORB to interact with the CORBA server and return the results.

The OA implements the adaptor design pattern, which is a way to make one interface work with another; also called a wrapper.

This communication is facilitated by an **object adaptor** (**OA**), which uses a skeleton to marshal an operation invocation into a language-specific operation (depending on the servant), and also provides a standard interface to an ORB. This means that the OA supports both ORB communication transparency and servant operation invocation, thus giving ORB implementers greater freedom without compromising the ability of servant objects to communicate with the ORB. The interfaces between the ORB and the OA vary from one ORB implementation to another.

In order for the servant to return a value to the client, the route is reversed: the value is communicated via the servant's skeleton, to the client's ORB and back through the client's stub, where data is unmarshalled into a form that the client machine understands.

A client can continue to use a reference indefinitely, but when the reference is no longer required, the client must request that the ORB releases the reference. Other operations involving references also need to be done through the ORB; for example, testing if a reference is equal to another reference.

By querying the ORB, via the object adaptor, a server can maintain a count of the links to servant objects and use this information to make decisions about the lifecycle of servant objects and about load balancing. This means that the OA also supports scalability. The server may control many servants and, if necessary, a server can create a servant on demand (a process that may take place without the client being aware that it has happened).

Requirements for lifecycle management of servant objects may differ from one domain to another and so there are various policies that an object adaptor can implement. Also, an ORB may provide more than one kind of object adaptor, designed for groups of objects that have similar requirements for lifecycle management, policies or concurrency requirements. For example, an object adaptor may make various types of object reference transient or persistent for a particular domain.

CORBA component model

In early CORBA specifications, there is no standard way to deploy a CORBA servant object or manage its lifecycle in a server. This has led to the development of the idea of the CORBA component as an extension and a specialisation of the CORBA object type.

A CORBA component can be described by a structure in an interface repository, as for a CORBA object, but it is deployed in a container that may support various qualities of service, much like an EJB component. The use of components allows for more predictable provision of services to objects and reduced complexity for developers through standardised interfaces.

Remote communication

Figure 3 illustrated communication using an ORB in a local context. If a local servant is not available, communication between a client and servant may take place over a network, as shown in Figure 4.

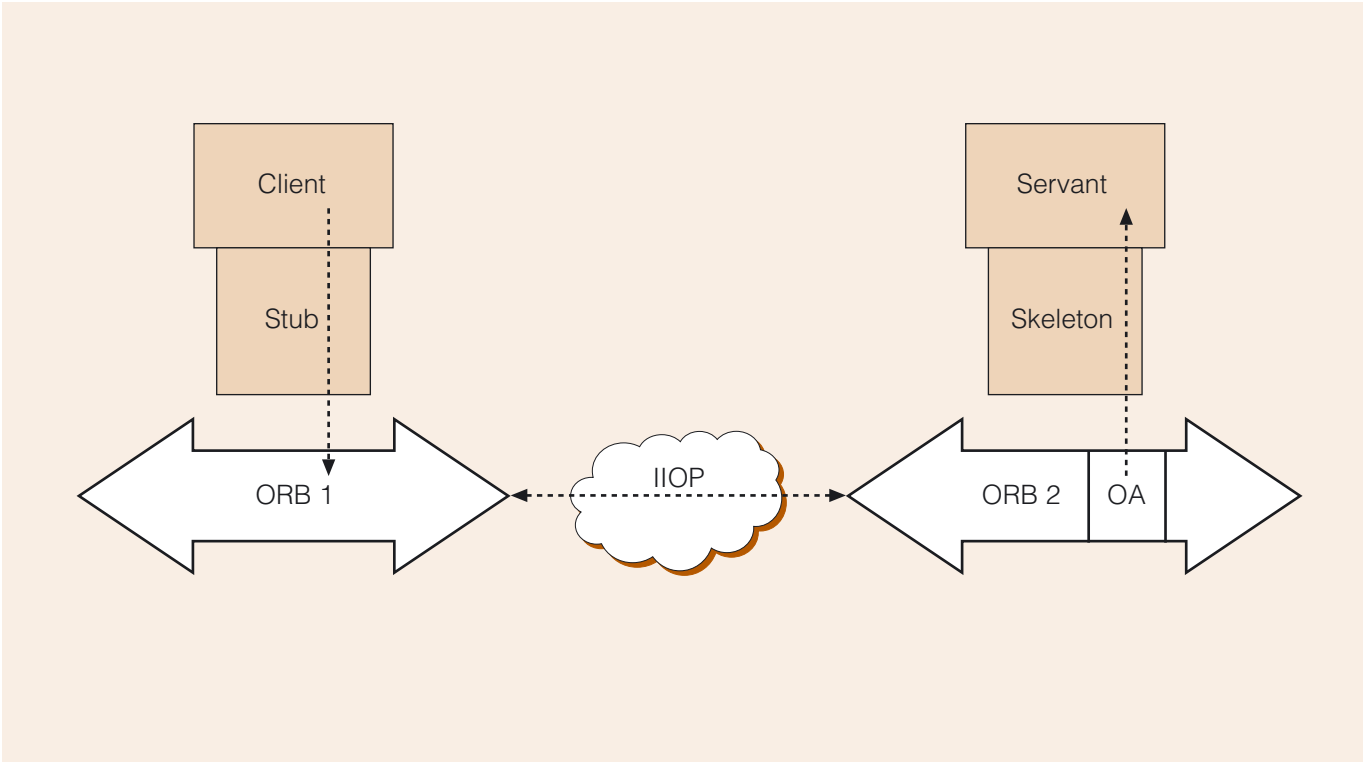


Figure 4 A remote ORB operation (a call to a servant)

On not finding a local servant, the client ORB can locate a suitable object using the **Internet Inter-ORB Protocol** (IIOP), which supports the CDR so that standardised object references and data types are transmitted. This protocol is used on top of TCP/IP-based communications to communicate with one or more remote ORBs to locate a server that can provide the required service. As before, a language-independent request arrives at the remote ORB, which forwards the request to the servant's object adaptor and on to the servant. A result is returned in a similar manner as it would be to a local servant, except for the additional step of communication between the remote and local ORBs.

IIOP does not specify a particular port for communication and this raises issues in how communications pass through firewalls.

There is no requirement for exclusive use of local or remote communication in a CORBA application – the location of the servant is transparent to the client. Client and server can be supported on the same platform or distributed.

Semantics of invocation

Passing by value is also possible.

If an operation involves passing CORBA object parameters, they are typically passed by reference; in this context, this means that the remote side gets a reference to an object, creates a local reference to it, and uses callbacks to alter the original object.

The normal semantics of invoking an operation via an ORB is **at-most-once**, that is, the code making the invocation will block and it will either successfully receive a result or fail due to an exception, resulting in a synchronous communication.

However, other semantics are possible, depending on the ORB implementation. For example:

- A deferred-synchronous semantics of execution is possible, where the client side returns immediately, but can later check to see if the server has left a result in a local callback handler.
- One-way semantics indicates that the ORB should do its best to invoke an operation, but invocation is not guaranteed and there is no server response.
- Asynchronous invocation can be used, with the client polling the server to determine when the server has the results for collection.

We mentioned earlier that the semantics of operations can be described within an interface. For example, if one-way semantics is required, the operation is specified as `oneway` in its interface. In the absence of any description, the operation is at-most-once.

CORBA therefore provides for a richer set of operation semantics than is generally built into programming languages, and this capability can be useful in distributed systems.

SAQ 5

(a) What is meant by saying that an ORB is middleware?

(b) What is the function of an ORB?

ANSWER.....

(a) An ORB is known as middleware because it is located between a client and the servant objects it wishes to invoke.

(b) An ORB provides facilities, similar to those of the Java RMI, that make it possible for application programs to send messages to and to receive messages from remote objects in the same way as for local objects and regardless of implementation language. The ORB locates a server able to provide a service and communicates with the server, returning results to the client if necessary.

Exercise 2

Distribution transparency refers to the notion that the users of a system may remain unaware of the fact that they are dealing with a distributed system. Besides this overarching transparency, there are other transparencies that tackle particular aspects of distribution transparency.

Use the internet and this unit to find out more about transparencies and in particular to find out what transparencies a CORBA-compliant ORB supports.

Discussion .....

The ORB supports the following transparencies:

- location – the location of a servant object is transparent to an object requesting a service;
- implementation – the implementation language of an object is hidden and the platform is not specified;
- object activation state – the client is not concerned with whether or not a server is actually running or a servant object has been instantiated;
- communication mechanism – the protocol used by ORBs to communicate with each other is transparent to an object requesting a service.

3.4 CORBA services

At the beginning of this section we mentioned the CORBA vision for objects being able to communicate with each other, no matter where, or on what type of machine, they are located. Having looked at some of the mechanics of how this is achieved, we now return to another aspect of this vision, namely the provision of common services that can be shared by components in distributed systems.

It is not possible to discuss all the services in the CORBA services specification (which is actually a collection of service specifications) here. These services are under continuing development. All services are described by interfaces, and implemented by servants, in the same way as application object services.

We have already mentioned the *naming service*, which allows a server to bind a name to an object reference and allows a client to resolve a reference, that is, to look up objects by name. This requires the client to know the correct name to use.

We present here a list of some of the other CORBA services.

- The *trading object service* allows objects to publish their interfaces to a repository and to be discovered by a client according to their purpose (for example, a client may require a printing service) rather than by their global name. This service also allows for scoping of service discovery using constraints.
- The *time service* includes reporting the current time within some margin of error, telling the order of events and telling the time between events. Timers are also provided to assist with event scheduling.
- The *concurrency service* provides operations to assist a server object in mediating between concurrent requests for access to servants, so as to maintain a consistent internal state. For example, locks of different granularity are provided and a server could dictate access for reading and writing to a file or variable. The client side does not need to be concerned with this detail.
- The *transaction service* supports transactions over multiple operation calls and across multiple objects, including concurrent operations. This is particularly important since failures may occur in a large variety of ways in distributed systems.
- The *persistent object service* provides interfaces for operations to do with maintaining the persistent state of objects.
- The *externalisation service* handles the serialisation of an object and subsequent reconstruction.

Naming services are sometimes known as ‘white pages’, and trading services as ‘yellow pages’.

- The *lifecycle service* manages the creation, copying, moving and removal of objects.
- The *collections service* supports treating objects as a group, comparison of objects and related tests such as uniqueness, and also iterating over a collection.
- The *event notification service* notifies objects that have registered an interest when a relevant event occurs.
- The *security service* includes authentication of users and objects, authorisation (access control), auditing and support to protect integrity of communications.

We have also mentioned that there are specialised domain service specifications for various domains, such as finance or telecommunications, as well as specifications for common service facilities such as email.

3.5 CORBA in use

The rise and fall of CORBA.

CORBA has been criticised for its complexity, but provides a rich set of services important for distributed systems. It is a mature platform, having been under development since 1991.

EJBs and CORBA

Clients not based on Java can use CORBA to communicate with EJBs using RMI over an IIOP connection. RMI-IIOP, as it is called, has been integrated in Java since version 1.3 and includes the full functionality of a CORBA ORB without the need for Java programmers to learn the CORBA IDL.

According to Sun Microsystems:

An example application using an EJB server with a CORBA client.

> RMI-IIOP combines the best features of Java Remote Method Invocation (RMI) with the best features of CORBA. RMI-IIOP speeds distributed application development by allowing developers to work completely in the Java programming language, writing remote interfaces in the Java programming language and implementing them simply using Java technology and the Java RMI APIs.
>
> (Sun Microsystems, 2002)

Implementations of this specification set are used in enterprise-level systems such as telecommunications. CORBA has been less successful in the internet arena, partly due to the complexity of the architecture, but also because of issues with specifications and its slow development in the face of other emerging technologies such as web services.

SAQ 6

(a) What is the purpose of the CORBA services specification in the OMA model?

(b) What is a trading service and how does it differ from a naming service?

ANSWER.....

(a) The CORBA services specification specifies various services that are of particular use in distributed systems, such as support for naming, transactions and security.

(b) A trading service allows clients to find objects based on the services they provide, whereas a naming service allows the lookup of objects based on their names.

Exercise 3

What are the main similarities and differences between the Java RMI and a system based on CORBA?

Discussion .....

Some similarities are as follows.

- Both RMI and CORBA implement an object model.
- Both provide location and communication transparency.
- Both use the mechanism of stubs and skeletons to communicate a call and to marshal arguments.
- The ORB provides a service similar to the RMI registry, where references to objects can be obtained by clients who wish to use them.
- Both CORBA and RMI support passing objects by value as well as passing objects by reference. (This means that RMI and CORBA both enable the sending of code to remote systems.)

Some differences are as follows.

- RMI is a programming technology. CORBA is more of an integration technology, occupying the spaces between programs, as it is a specification for interaction between objects.
- RMI is portable to any platform that supports a JVM. CORBA runs on any platform with a CORBA-compliant implementation.
- CORBA is a much more extensive system than RMI, and offers additional services commonly required by distributed systems as well as domain-specific services.
- RMI could be considered to be a homogeneous middleware, in that it is purely Java based. A major advantage of CORBA is that an implementation is potentially language-independent (heterogeneous) and this provides an ability to integrate legacy services.
- In RMI the interface is necessarily in Java, while in CORBA the interface is defined using the language-independent CORBA IDL.
- RMI is a synchronous form of communication, while CORBA provides for more varied forms of interaction.

You could use the Java native-code interface with RMI to link to objects written in other languages.

4 SOAP

We are now going to consider a more lightweight approach to distributed system communication based on the protocol **SOAP**. (SOAP dates from 2000 and is a standard maintained by the World Wide Web Consortium.) Like CORBA, this protocol supports communication between heterogeneous systems, but in this case the systems must have the ability to process SOAP messages.

Although it is still often referred to as Simple Object Access Protocol, in fact this acronym has been dropped and SOAP is not actually object oriented (or even particularly simple). Rather it is typically built on top of HTTP and involves sending messages that have a standard form, using XML syntax.

SOAP does not involve the overhead of ORB communication, but also does not provide the kind of support for multiple implementation languages that CORBA does or the range of services that CORBA provides. Nevertheless, SOAP has gained considerable popularity, because:

- it is the standard way of implementing web services and a popular approach to producing web-based software services, which we discuss in Section 5;
- XML-based messages are platform independent, and this aids interoperability.

We will see other reasons for SOAP's popularity in Subsection 4.2, but before we discuss SOAP, we need to give you some more understanding of XML.

4.1 XML

XML is a form of SGML – Standard Generalised Markup Language.

XML (**eXtensible Markup Language**) is a way of providing information about the text in a plain text document; that is, it allows a semantic markup of text. The language is also under continuing development by the World Wide Web Consortium (W3C).

You will be familiar with the idea of a markup language from the hypertext markup language (HTML) used to deliver web pages, which uses paired tags as in

```
<bold>my bold text</bold>
```

as a way of attaching some information to any text between them. The tag type can be used to interpret the enclosed text in some way, but does not dictate exactly how this happens. For example, the HTML bold tag can be used by a browser to render the enclosed text in a bold font, but the procedure for doing this is not specified. XML applies the same technique to indicate the *meaning* of data as opposed to its *presentation*.

Like other markup languages, XML provides a set of syntactic conventions, such as the use of angle brackets to delimit tag names. An XML message must conform to this syntax and so it is possible to determine whether a message is **well formed** (syntactically correct) or not.

Because individual platforms may use non-portable characters, platforms must provide mappings to translate to and from Unicode.

XML documents are portable in that they can be written in Unicode, which also supports internationalisation of software. Because it is text-based, XML is also readable by humans and compresses well.

XML refers to the semantic markup of text as an element. An **element** is a pair of tags enclosing some text and possibly also encloses other elements. Unlike in HTML, where there is a fixed set of valid tags, XML is extensible, meaning that an author may define new element names and sets of valid element names may be defined for particular domains.

The idea of semantic markup is very general and can be used in all manner of ways to determine the processing of enclosed text. For example, an XML interpreting program may choose to ignore certain elements, to search for information within certain elements, to save information marked up a certain way, or to execute methods named in elements. However, semantic markup inevitably means that more information is transmitted than might otherwise have been required had the communicating parties agreed beforehand on the meaning attached to all message data.

XML has sufficient advantages that it has become a de facto standard for middleware communication and there are therefore many tools available for processing XML data. Apart from its use as the SOAP message format, XML is being used as a database storage format and as a standard to represent metadata (data about data) as seen in deployment descriptors in Java EE.

An example document

Below is an example of text you might see in an XML document.

```
<?xml version="1.1"?>
<shop name="Flutes R Us" type="music">
  <special_offers>
    <flute>
      <description>Miltcher Deluxe</description>
      <condition>new</condition>
      <price>120</price>
    </flute>
    <flute>
      <description>Yikoban wooden</description>
      <condition>battered</condition>
      <price>20</price>
    </flute>
  </special_offers>
</shop>
```

Prologue

Just as an HTML document is identified by the `<html>` tag, an XML document must be identified by an initial declaration, known as a **prologue**, which identifies the document as XML and declares its version number. This is the first line of the example.

The first line is enclosed in opening and closing angle brackets with question marks, which indicates a processing instruction for the recipient. (In XML 1.0 this declaration is optional.) It is also possible for this element to include other information about the document, such as the character encoding it uses.

Body

Following the prologue is the body of the document itself. There are six different kinds of element represented in the body; the first of which is `shop` and the last `price`. Note that the element names here could have been invented by the author of the document, and this means that the sender and recipient must agree on the semantics of each tag.

The first element is called the **root element**, which is an outer envelope within which the rest of the message can be formed.

The same attribute names could be used with other elements. Names have scope.

We also have shown two **attributes** (`name` and `type`) which provide more information about the `shop` element and can be used by the recipient of the message in determining the processing to be carried out. Attributes are similar to instance data in a class.

As we will only be reading XML and not writing it, we will not concern ourselves with style rules regarding when data is better defined using an attribute or an element. We have shown examples of attributes here as you may encounter them when examining XML files.

As an example of the kind of processing that could take place on our example document, if our shop appeared in a `<shopping_mall>` including other kinds of shops, the interpreting program could go through the document and present all music shops in some way, or find all special offers, using both attribute and element data to determine the processing that takes place.

It should be clear that XML provides a very general way of adding value to plain text, while incurring some overhead due to the descriptive markup information that is included.

Some other features worth noting are:

- the elements in XML are case sensitive;
- rather than being enclosed in a pair of tags, an element may have only one, ended by the text `/>`, known as a self-closing tag;
- XML uses similar techniques to HTML in order to represent characters such as > (`&lt;`) and < (`&gt;`);
- as in HTML, comments can appear between the tags `<!--` and `-->`.

Validating documents

Given that anyone can define element names it is important that there is a way of describing an acceptable markup for a particular domain.

The prologue can optionally include a DTD.

One way of doing this is the use of a **document type definition** (**DTD**). A DTD can describe not only valid kinds of element, but also their relationships to each other, and valid values of attributes for documents with a particular root element.

So, in our example, a DTD for a `shop` document may state that the `price` element must be defined and that within a shop a `condition` may occur only within a `special_offer`.

If an XML document is well formed and conforms to a DTD, it is said to be **valid**.

There are also standard DTDs within particular domains, created by committees and discussion groups. An example is the Textual Encoding Initiative (TEI) markup, which is used for annotating academic works. There are also standard markups for the description of mathematics (MathML), speech (VoiceXML) and scalable vector graphics (SVG XML). Other domains with standard DTDs include health care and the automotive industry.

Different DTDs therefore can be used to communicate specific kinds of message between applications according to their domain, and a recipient can check that a message conforms to an expected structure.

Schemas

DTDs provide only a limited ability to constrain documents. A more powerful alternative to the DTD is the **XML Schema Definition** (**XSD**), a W3C standard. An XML schema is itself an XML document, and so can be validated and processed using standard tools.

Although schema validation can be used in addition to DTD validation, schemas address some deficiencies in DTDs. For example, schemas support the expression of constraints on data types in elements (for example, numeric types) and the use of ISO codes, such as en-GB (British English), which allows for better type checking and more rigorous constraints on XML documents.

Schemas also support a more object-oriented description of XML document instances, as they may inherit from each other or incorporate user-defined types.

Finally, it is possible to automatically generate a schema file from an XML file, although doing so may not capture all the rules you require.

Processing

Given XML's popularity, tools have evolved for checking whether XML documents are well formed and valid and for accessing their contents.

The **Document Object Model** (**DOM**) is a platform-independent standard defining how to read and manipulate an XML document as an object representing a tree data structure and so is suited to XML validation. Once the document is loaded, the data in the document can be accessed in any order.

The **Simple API for XML** (SAX) is a way of viewing XML as a stream of data, which is useful when validation is not required. As one element after another is parsed, SAX can deliver the information as 'events' within the document; that is, it can deliver elements as they are read (serially). This is naturally faster than loading the whole document, but will only be suitable for some applications, as it does not support random access to elements.

Web services and other environments that rely on XML will provide sets of tools to process XML documents using approaches such as DOM or SAX. For example, in Java you will find an API for XML processing called JAXP.

Namespaces

Because an author can create their own element types and attribute names, it is quite possible for these to be used in different contexts with different meanings. As you have seen elsewhere in the course, while in a local system naming conflicts could be resolved, in a distributed system a way of scoping names is required.

An XML **namespace** is a way of identifying a set of names and is declared as an attribute of an element. There is a predefined (reserved) attribute called `xmlns` (XML namespace) for this purpose.

A namespace is similar to a `package` in Java.

For example, consider that the element `<flute>` may appear both as a description of a musical instrument and as a description of a drinking glass, perhaps. We would then need a way of telling what kind of flute we were referring to in an XML document that would uniquely identify the namespace in a distributed context.

Uniform resource identifiers are a superset of URLs.

There is a syntax for creating namespaces in the root element of a document using the special `xmlns` attribute and a uniform resource identifier (URI).

```
<flute xmlns="http://www.musicshop.com">
  <description>Yikoban wooden</description>
  <condition>new</condition>
  <price>120</price>
</flute>
```

This identifies the namespace for the document using the (case-sensitive) URI value `http://www.musicshop.com`. A flute belonging to a different set of names could be distinguished by use of a different URI attribute value. Also, this identifies that the `description`, `condition` and `price` elements are within the same namespace, because they are within the scope of the `flute` element. Names within a namespace must be unique.

Association with a specific URI, typically a URL, is the standard way of scoping names. The URI itself is simply a way of placing names within an existing namespace, although the URI location may contain information about the namespace that can be used by humans to interpret it. URLs are often used because they are already unique.

Optionally, string names can be used to identify namespaces, as in the following fragment.

```
<Shop xmlns:musicShop=http://www.musicshop.com
      xmlns:homeShop=http://www.musicshop.com/catalogue>
```

Names beginning with `xml` are reserved and the prefix `xml` represents a predefined namespace associated with w3.org.

This provides two namespaces that can be referred to by the names `musicShop` and `homeShop`, known as **namespace prefixes**. Elements in these namespaces can be distinguished from each other using the prefix in front of a **simple name** to form a **qualified name**. This style of namespace declaration also helps provide meaningful names for human readers.

This ability to scope names means that, for example, if you are dealing with a document that has both a drinking-glass `flute` and a musical `flute` in it, you could scope both names.

For example, we could now have the following descriptions within the same document.

```
<musicShop:flute>
  <description>Yikoban wooden</description>
</musicShop:flute>

<homeShop:flute>
  <description>Green glass</description>
</homeShop:flute>
```

You then might decide to process only those flutes that are in the `musicShop` namespace.

It should be clear, now that we have a smattering of XML, how this message format can be used for heterogeneous communications. Provided that the communicating parties can process XML, have a shared idea of what a document means and can communicate over some channel (such as HTTP), they can communicate, regardless of the platform or the software implementation language.

As XML is used in SOAP messaging, the same advantages are often quoted in support of SOAP.

4.2 SOAP messages

XML can be used with or without the support of a protocol such as SOAP, but SOAP provides a structure to such communication.

- SOAP provides scope for sending complex information such as structures and objects, and their data, including user-defined data types.
- Communicating nodes may take on one or more roles so that they can decide how they should act on the contents of a message, and role information may also be passed within a message.
- SOAP automates marshalling and unmarshalling of method parameters and return values, thus supporting the development of APIs for web-based components (which we call web services).
- SOAP incorporates an exception handling facility, which would have to be manually programmed if using XML over HTTP without SOAP.

There is a price to pay in terms of an overhead in both communication and complexity (as compared to plain XML). SOAP may actually be slower than CORBA's IIOP because of the extra text transmitted and because of marshalling to and from XML.

A SOAP message must conform to some requirements; for example, it must be well formed and meet some namespace restrictions (such as using a URI to qualify names). The contents of such a message are also described using XML; however, the semantics of the message markup are defined not by SOAP, but by the communicating parties, and so SOAP provides an extensible and adaptable form of communication.

In SOAP, there is no specification of what protocol should be used to transmit a message, but it is almost always HTTP. Sometimes SMTP (Simple Mail Transfer Protocol) is preferred. This means that desirable distributed systems properties, such as security and reliability of communication, are not implemented by SOAP, but may be supported by the underlying protocol, or by standardised SOAP extensions. We will only consider HTTP-based SOAP messages.

SOAP – digital signing.

A particular advantage of using HTTP is that firewalls are usually configured to allow traffic via the normal HTTP port. Of course, there are also risks attached to this form of tunnelling, in that a SOAP message may become a route for an attacker to gain entry to a system.

Recall from *Unit 9* that HTTP is a stateless, connection-oriented, request–response protocol for web communication. A client (typically a web browser) sends a request, and a web server returns a response.

State can be introduced through techniques such as cookies.

The keywords `GET` and `HEAD` indicate requests for data from the server, while `POST` and `PUT` are ways of sending data to the server. A request or response involves using one of these keywords together with a URL. SOAP using HTTP involves sending a SOAP message as the request or response body.

More complex interactions can be implemented by SOAP applications keeping track of a 'conversation' between the two end points of the communication.

4.3 SOAP message format

Messages can be encrypted for security.

A SOAP message will consist of an outer SOAP envelope that contains information about the sender and intended recipient, and information about the message such as how it should be processed. Within the envelope is an optional header and a required body.

Figure 5 shows the hierarchical relationships between the parts of a SOAP message.

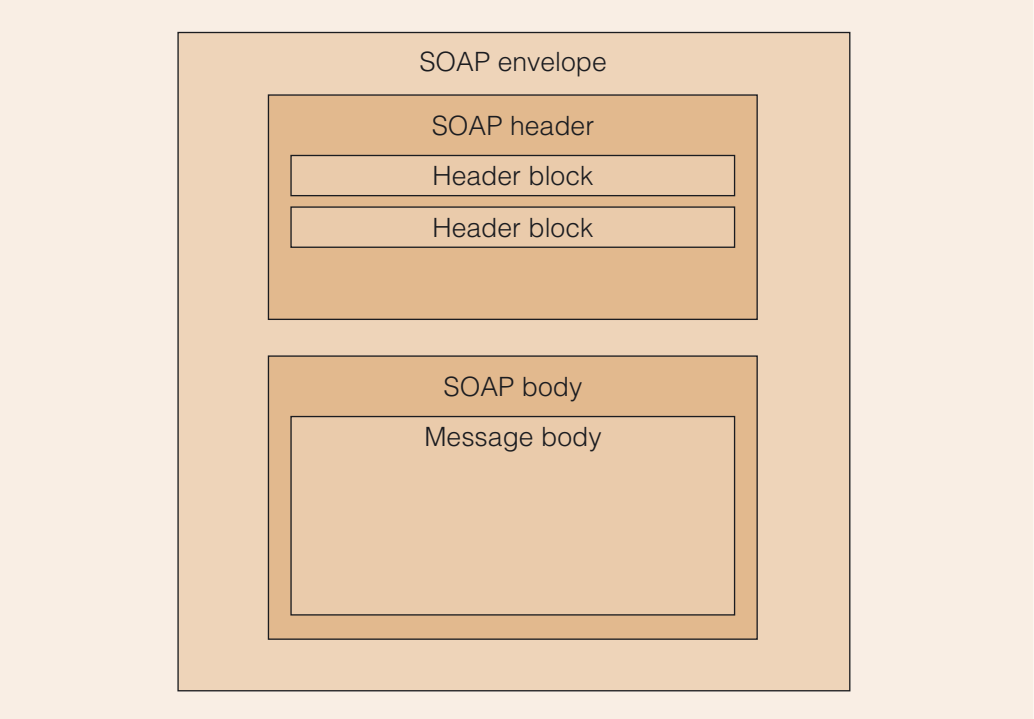


Figure 5 A SOAP message with optional and required elements

The header section can include contextual information, for example authentication information, or a time-stamp.

There is also a `fault` element type for reporting errors.

There are predefined element and attribute types for SOAP messages, including an `Envelope` element and a `Body` element.

An example `Envelope` element is shown below.

```
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    <!--
      The message body goes here (see example below).
    -->
</soapenv:Envelope>
```

In this case, the `Envelope` element is within the `soapenv` namespace.

SOAP request and response

By embedding a request in a soap envelope, a client can request that a server performs some action.

A SOAP envelope can be enclosed in an HTTP request, for example, as illustrated in Figure 6, which shows a typical remote procedure call use of SOAP. The dashed line shows the responsibility of a SOAP package supporting the SOAP protocol in this case.

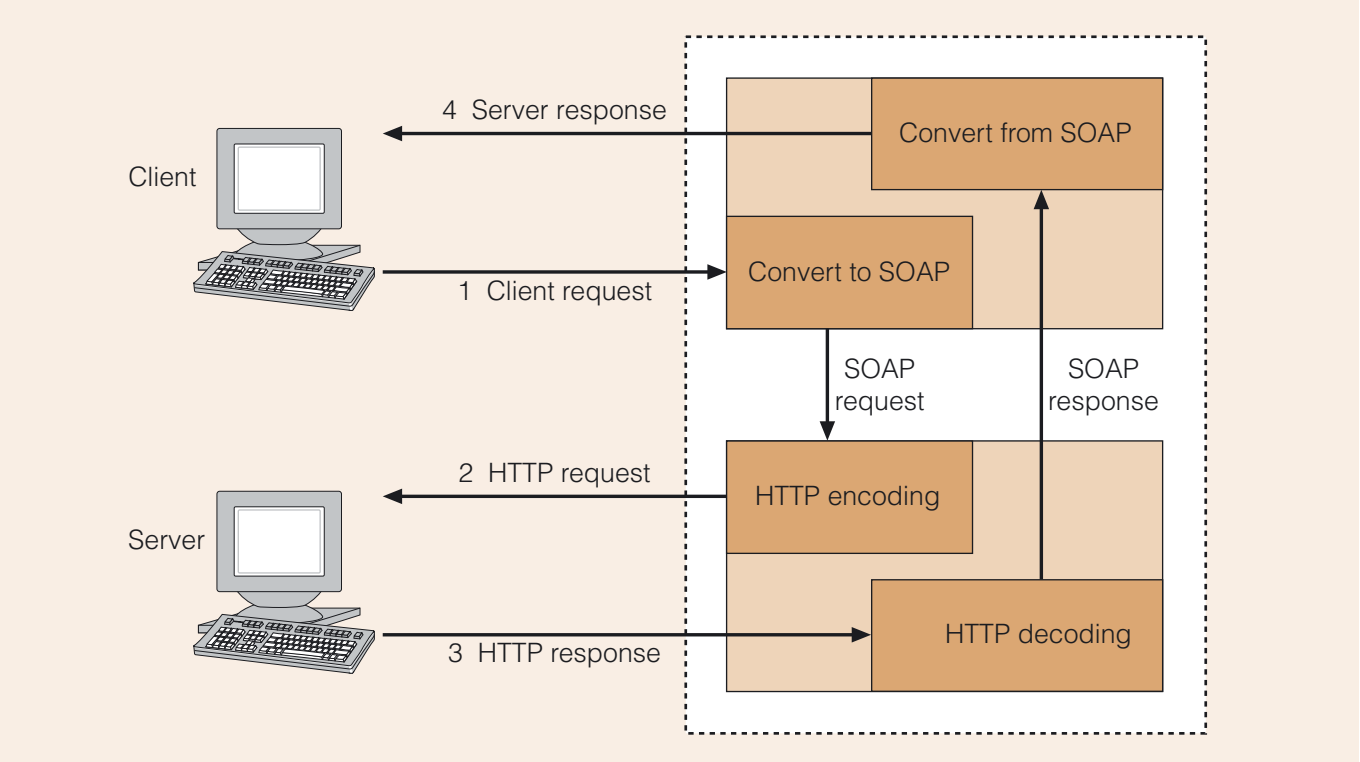


Figure 6 An example SOAP call

Figure 6 shows the normal sequence of a SOAP request and response. A call to a service is marshalled into an XML message in SOAP format. The SOAP message is then wrapped in an HTTP request and sent to a server. The server returns an HTTP response, which is unwrapped to retrieve a SOAP response and then unmarshalled for return to the client. A SOAP implementation is responsible for conversion to and from SOAP message format and for handling the HTTP request and response.

The client request might be sent using the `POST` method, as illustrated in Table 1.

Table 1 Example SOAP request

Text	Comment
`POST /SomeURL HTTP/1.1`	HTTP request
`Host: http://somehost.com` `Content-Type: text/xml; charset="utf-8"`	HTTP header lines
`SOAPAction:"/adding"`	Start HTTP request
`<?xml version="1.0" encoding="UTF-8"?>`	XML header

`<soapenv:Envelope`
`xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"`
`xmlns:xsd="http://www.w3.org/2001/XMLSchema"`
`xmlns:ns1="http://act11_2/">`
	Start SOAP envelope

`  <soapenv:Body>`	Start SOAP body

`    <ns1:addUp>`
`      <num1>3</num1>`
`      <num2>2</num2>`
`    </ns1:addUp>`
	SOAP body (message)

`  </soapenv:Body>`	End SOAP body
`</soapenv:Envelope>`	End SOAP envelope; end HTTP request

A SOAP server would receive the SOAP envelope via HTTP and process it. The `SOAPAction` value can be used by the server to decide how to handle the request. For example, it could be used to specify a remote object to which the request is directed (for example, a servlet), it could represent a URL, or it could be used by a firewall to filter the data to a server dealing with SOAP requests.

The key point here is that the SOAP body can be interpreted by a server in a variety of ways, including that the name `addUp` could represent a method. The elements `num1` and `num2` could demarcate the arguments to this method, for example.

Let us suppose that in our example the request actually refers to an operation with a header `int addUp(int num1, int num2)` that returns the sum of `num1` and `num2`. Then the server could return a response via HTTP something like the following.

```
HTTP/1.1 200 OK
Content-type: text/xml charset=utf-8

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:ns1="http://act11_2/">
  <soapenv:Body>
    <ns1:addUpResponse>
      <return>5</return>
    </ns1:addUpResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

Alternatively, the response body might also indicate that something was wrong by returning a fault of some sort (in a `fault` element).

Whatever the server returns, it can be interpreted by the client and used as needed.

Clearly, generating SOAP messages directly from code would be tedious and easy to get wrong, so there are tools supporting this kind of request and response generation.

The current version of SOAP provides for a large variety of synchronous and asynchronous usage scenarios, depending on the transport protocol used. For example:

Other SOAP usage scenarios.

- one-way message passing ('fire and forget'), where an unacknowledged message is sent to a recipient;
- asynchronous messaging, where the response is tagged for identification with an earlier request;
- remote procedure call (RPC);
- event notification ('pull'), where a client subscribes to receive messages about events.

5 Web services

The SOAP communication mechanism is a useful and lightweight approach to communication over the Web, allowing us to use the internet as a computing resource as well as an information resource.

However, SOAP is weaker than CORBA and RMI in that it does not provide a registry to store information about services and a means of discovering information about these services. In this section we discuss how this extra functionality is provided by the infrastructure for web services.

The idea of **web services** is that businesses or other providers can implement code on their chosen platform and then register services provided by their code with online registries, so that it is possible to search for relevant services.

Web services are extensible in that they may call on other services for part of their business, in the process possibly sharing information about the user or the context of the interaction taking place.

CORBA versus web services.

The main requirements for web services are:

- a way to describe the nature of the services in terms of an API;
- a way to register a service in a repository so that it is findable;
- a way to find a service;
- a mechanism for a client and a service provider to communicate.

This is not a new idea, and the architecture is not dissimilar to that used by RPC, RMI and CORBA objects. Web services, however, assume use of XML as the format for messages, and are particularly suited towards use over HTTP and to loose coupling between service providers and clients. The use of HTTP also means that, as for SOAP messaging, web services are typically stateless and designed so that they do not require a memory of previous interactions with a client.

It should be no surprise that the typical web services communication mechanism is SOAP, as this greatly simplifies the remaining work to be done. In addition, the use of XML allows use of existing tools, for example, for schema checking, which is useful in addressing security through type checking.

In the following subsections we will be discussing the remaining pieces of the puzzle. The standard for the description of web services is called the **Web Services Description Language** (**WSDL**). WSDL is also a component of a standard for service description and discovery called **Universal Description, Discovery and Integration** (**UDDI**).

5.1 Overview

A web service is a component delivering a computing resource via the Web and is an example of a service-oriented approach to communication. Instead of requesting a document using a URL, you can request a service from a web service using a URI. This means that web services are another way of accessing heterogeneous systems transparently, with the Web as the underlying medium.

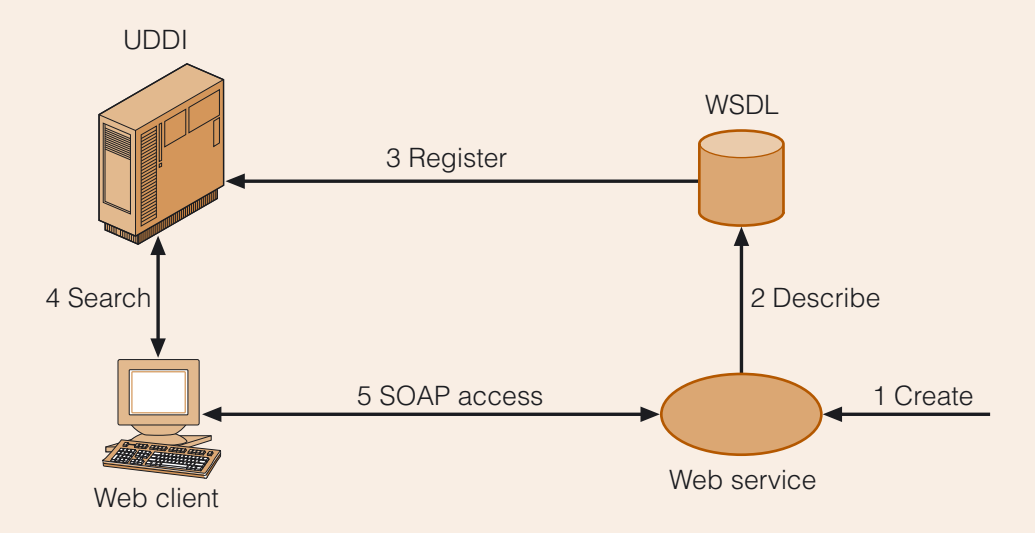


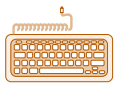
Figure 7 A typical web service

Figure 7 illustrates a typical web service and its relationship to a client in five steps.

1 The web service is created by a provider.
2 The service is described using WSDL.
3 The service is registered with a UDDI server using WSDL.
4 A client can query the UDDI server to find out how to use the service.
5 The client can then access the web service correctly at the provider.

The provider may implement the web service in any form, provided that the component can communicate using SOAP messages.

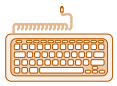
For illustration, we will use the adding-up-numbers operation of our SOAP example from the previous section. Java provides annotations that allow you to expose a method as a web service, and in Activity 11.2 we ask you to implement an adding-up service as a Java method as follows.



Activity 11.2
Writing a web service.

```
@WebMethod
public int addUp(@WebParam(name="num1") int num1,
                 @WebParam(name="num2") int num2)
{
  return num1 + num2;
}
```

The annotations were automatically generated when we requested that this method be made available as a web service, so we need not concern ourselves with their details. All we need to know is that the service offered by `addUp` has been marked for delivery as a web service using the `@WebMethod` annotation.



Activity 11.3
Writing a web service client.

Once such a service has been implemented, it can be described using WSDL and the service provider can register the service with a repository, for example using UDDI, which we will discuss in Subsection 5.3. The client can query the repository to discover the service.

Such a service can be accessed in a variety of ways, for example, using a custom graphical user interface. In this case it could look like a calculator, with number buttons and a + button on it. Alternatively, a web service can be accessed using a web browser and web forms.

NetBeans provides a facility to generate web forms automatically, based on the service, as illustrated in Figure 8 for the `addUp` service. In this case, we implemented the `addUp` method in a package called `calculator` and in a class called `AddUp`.

Figure 8 A web service accessed using a browser

In this case, the correct URL in a browser allows access to the service. A particular service can also be requested from a client application using its name, or a client may use a repository such as that specified by UDDI to determine whether a suitable service exists. The WSDL description stored in the registry provides the information the client needs in order to know how to invoke the service.

A web service description may need to indicate more than the signatures of the operations it provides. One reason for this is that web services may be provided by one business for use by another business – thus there may be other metadata, such as terms and conditions of use, required in a web service description.

In addition, service users may require further qualities, such as security and support for transactions. There are web service specifications that address such issues. For example, there is a security protocol specification produced by the OASIS consortium incorporating standards for encryption of XML messages and describing the use of signatures in SOAP communications. Other similar specifications have come from the W3C group. As such, the simple approach we have suggested here is open to further development and can be made more robust – however, we hope this introduction to the subject has conveyed how simple this approach to distributed communication can be to implement and use.

SAQ 7

Are web services object oriented?

ANSWER.....

As web services are SOAP-based, they are not object oriented, although object-oriented wrappers can be used to encapsulate the services. They are better described as service oriented or message oriented.

5.2 WSDL

As in CORBA, there is a platform-independent way of describing the interfaces to web services. In this case it is known as the Web Services Description Language (WSDL), which is an XML markup describing the operations provided by a web service component.

Just like the interfaces we have seen previously in Java and CORBA, WSDL is a way of separating the implementation of a component from its description. It will describe:

- the location of a service (host and port);
- the protocol to be used;
- the messages used, the types of argument and the return types.

That is, a WSDL description defines the nature of an acceptable SOAP message for a web service.

In order to make our adding-up-numbers service available, it requires a WSDL description, which can be created by hand or from an existing implementation using a WSDL generation tool. It is more likely that a description will be required for an existing service, perhaps for a legacy application, or to provide an existing service to clients using the Web as the medium.

Automatic generation of WSDL requires a mapping from the implementation language to the WSDL. Mappings exist for Java and languages used by the .NET platform, for example. This means that web services are also language-independent, provided that a language mapping is available to WSDL.

As messages are SOAP-based, there is support for all the types included in the SOAP protocol.

The following are some examples for Java to WSDL mapping, which you can see reflected in the WSDL file below.

- A method maps to an `operation`.
- A parameter maps to an `input`.
- A return value maps to an `output`.
- A throws clause maps to a `fault`.

The automatically generated WSDL file for the `addUp` operation we have just described is shown below. The operation is implemented in the class `AddUp` in the package `calculator` (notice that the package in Java maps to a namespace).

```
<definitions targetNamespace="http://calculator/" name="AddUpService">
  <types>
    <xsd:schema>
      <xsd:import namespace=http://calculator/
        schemaLocation="http://PCMA417.open.ac.uk:8080/AddUpService/
        __container$publishing$subctx/WEB-
        INF/wsdl/AddUpService_schema1.xsd"/>
    </xsd:schema>
  </types>

  <message name="addUp">
    <part name="parameters" element="tns:addUp"/>
  </message>

  <message name="addUpResponse">
    <part name="parameters" element="tns:addUpResponse"/>
  </message>

  <portType name="AddUp">
    <operation name="addUp">
      <input message="tns:addUp"/>
      <output message="tns:addUpResponse"/>
    </operation>
  </portType>

  <binding name="AddUpPortBinding" type="tns:Echo">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http"
      style="document"/>
    <operation name="addUp">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>

  <service name="AddUpService">
    <port name="AddUpPort" binding="tns:AddUpPortBinding">
      <soap:address location="http://PCMA417.open.ac.uk:8080/
        AddUpService"/>
    </port>
  </service>
</definitions>
```

Fortunately, we need not concern ourselves too much with all the detail in this description. The important point is that it is written in a standardised form of XML and this allows a client to query a registry that will have this information, and thereby discover how to access the `AddUp` service and invoke the `addUp` operation (and any other operations this service defines – in this case there are none).

You have already seen examples of SOAP messages that would be used in operating this web service in Section 4. Nothing has changed here, except that now we can easily make our operations available for use by SOAP messaging and allow their discovery by clients.

The possible semantics of invocation for web service operations are the same as those for SOAP, so this remains a flexible form of remote communication.

5.3 UDDI

Universal Description, Discovery and Integration is a registry project started by a consortium known as OASIS in 2000 to provide a platform-independent way for web service providers to publish information about their web services on the internet. This is an idea similar to an interface repository in CORBA or a registry in RMI.

Once a service is registered, service discovery can take place from the client side.

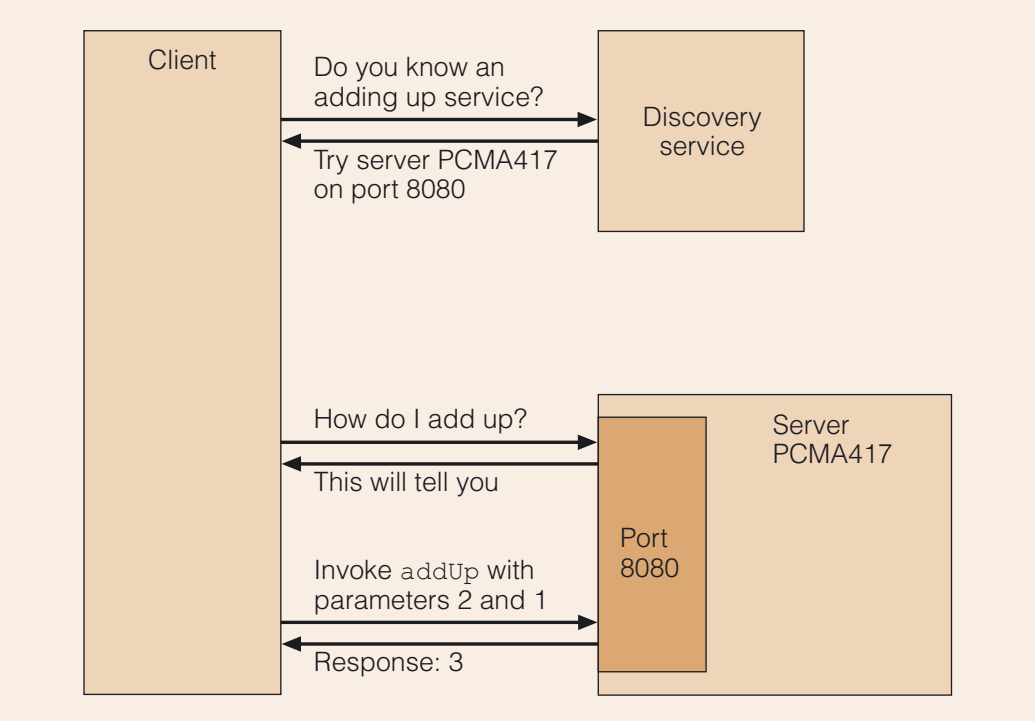


Figure 9 A UDDI lookup and web service

UDDI provides a standard way of publishing services so that it becomes feasible for businesses to find new customers or to find businesses that provide the services they require. UDDI is itself implemented as a web service, so it is queried using SOAP messages.

A UDDI entry consists of XML-based white pages (free text), yellow pages (organised according to the semantics of operations) and green pages (specifications of services).

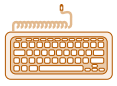
This means that UDDI supports customer-to-business and business-to-business communication. In addition, this kind of lookup service supports the integration of software within a business by promoting a service-oriented approach, which facilitates communication between parts of business systems.

5.4 Uses of web services

Various companies have built web application servers that provide a unified platform integrating UDDI, WSDL and SOAP support (for example, IBM's WebSphere). Sun Microsystems provides the Java Web Services Developer Pack (Java WSDP), incorporating the JAX-WS API and the Java APIs for XML.

In addition, frameworks such as .NET and Java EE are adding the kind of support for security, authentication and transactions that is lacking from the simple idea of web services. Such support will increase the chances of web services being used between businesses, rather than being merely used within a business to enable communication between its own heterogeneous systems.

A web service **container** (as provided in Java EE and .NET) takes care of the details of XML communication and load balancing, and provides support for transactions, freeing the programmer to concentrate on the business logic.



Activity 11.4
Accessing an online web service.

There are some notable examples of web services, perhaps most significant of which is credit card processing. Other examples are weather forecasts and stock market figure services. However, the explosion of web services that was once predicted has not come to pass (Platt, 2004).

Some major UDDI business registries have closed down recently amid suggestions that the publishing and discovery of software services is not a good fit to the way that businesses operate. However, interest in deploying web services remains high.

Microsoft, IBM and SAP discontinue UDDI registry effort.

Business processes and BPEL

To make web services more useful for business interaction, it is important to be able to describe business process interactions in a formal and abstract way. An abstract view is important to separate the logic from the implementation and also so as not to reveal more than is necessary about the business logic.

WSDL does not describe state, whereas business interactions typically require expression of state over several exchanges. This means that the needs of a language to describe business processes include:

- the ability to describe data-dependent behaviour such as branching and loops;
- specification of exceptions and their handling;
- the ability to nest descriptions to build up more complex descriptions.

This has lead to the development of various languages such as the Business Process Execution Language (BPEL), written in XML. This performs a similar function to features of programming languages that describe object-oriented features, in that they describe legal interactions between parts of a system at a high level. This enables BPEL to act as a wrapper for web services.

Various environments provide assistance with producing BPEL code, for example NetBeans has a plug-in BPEL visual designer to help build web services in a BPEL process.

Web services and, more particularly, web services as part of a larger framework such as .NET or Java EE, are seen as one solution to the issue of enterprise application integration, which simply means that they are systems for making disparate enterprise level systems work together.

Requirements for enterprise application integration are suggested by Clark (2005):

- a platform-independent, non-proprietary interface, accessible from anywhere;
- an open-standard communication language;
- ability to track requests sent to the system;
- good security/flexible user authorisation/support for user roles;
- extensible functionality;
- support for transactions and batch processing;
- support for validation of data;
- ability to define business rules;
- support for report generation;
- administrative function support (such as suspending a transaction);
- robustness/error reporting.

It is worth noting that the web-based interface is a natural solution to the first requirement in this list, as XML is for the second, and this may account for some of the excitement surrounding web services, as well as interest in enterprise environments such as .NET and Java EE, which support other enterprise integration requirements. In the next section, we take a closer look at .NET and Java EE with a view to showing how some of these requirements are fulfilled.

6 Java EE versus .NET

In this section we will give an overview of the components of .NET and Java EE platforms and highlight some of their key similarities and differences. In this section we are referring to the enterprise version of .NET.

There are versions of .NET targeted at small devices – similar to Java ME in the case of Java.

Both .NET and Java EE claim to speed up and simplify enterprise software development. As such, they both support heterogeneous, distributed communication, either in a local context (intranet) or via the internet.

The first point worth noting is that Java EE is a specification, and vendors can produce applications that comply with that specification, so there may be a choice of vendors of compliant applications, including middleware and application servers. On the other hand, .NET is a framework managed by Microsoft, incorporating a set of technologies for writing and running software.

Sun Microsystems has released source code for Java EE under a general public licence (GPL), whereas the standard implementation of .NET (built into various versions of Microsoft Windows) is not open source. Microsoft has, however, released source code for the major component of .NET under its own licence known as the Microsoft Reference License (which forbids commercial use), as well as a binary version of .NET for FreeBSD and Mac OS X. There is also the ongoing development by Novell of an open source .NET environment for various platforms including Linux, called Mono, based on the published .NET specifications.

.NET is available as part of the most popular IDE for building .NET applications, Visual Studio .NET. However, just as for Java EE, technically all you need is the relevant software development kit and a text editor. There are command-line tools that you can use to build your software, if you want to work that way.

It is also possible for Java EE and .NET systems to interact with each other, and to employ other technologies such as CORBA as a solution to distributed communication requirements. Both platforms also support web services, and as such have support for SOAP messaging and XML processing built in.

6.1 The history of .NET and Java

The development of Java by Sun Microsystems dates back to the early 1990s, at which point it was licensed rather than open source. This ‘write once run anywhere’ software quickly gained ground. Sun Microsystems then licensed Java to Microsoft, which went on to develop its own version of the Java Virtual Machine (MSJVM) and its own version of Java called J++. Lawsuits between Sun Microsystems and Microsoft followed. The results of these cases are still unfolding at the time of writing and Microsoft is set to cease support for MSJVM (which was bundled with early editions of Windows XP) at the end of 2007.

To some extent, the development of .NET, which was announced in 2000, was precipitated by the breakdown in relationships between Sun Microsystems and Microsoft, as well as by the success of Java. However, in its early days, .NET was seen by many writers as a marketing exercise, and as a matter of Microsoft having introduced an umbrella term for a number of existing technologies.

There is still a good deal of rivalry between the two camps, with developers on the Java side sometimes taking an anti-Microsoft stance and with some .NET advocates regarding Java supporters as driven by anti-Microsoft feelings rather than by thoughtful judgement as to which of the two frameworks is better for a given job.

The .NET framework has matured considerably since 2000 and is at version 3.5 at the time of writing.

These days, .NET is more associated with C#, a Java-like language, VB.NET (an adaption of Visual Basic), J# (Microsoft's later version of J++), COBOL, C++ and JScript (a Microsoft version of JavaScript). Other languages such as Eiffel, Perl and Smalltalk are also supported.

COM, DCOM and COM+

The Component Object Model (COM) was an early Microsoft technology to allow software components to communicate with one another within a platform. COM describes a binary specification and COM objects implement an interface to expose the services they provide to clients. These clients are themselves described through the implementation of IDL interfaces, with namespaces that cannot interfere with one another (so that if two COM objects provide a service with the same name, they are distinguishable from each other). Because the interface between components is described at a binary level, implementation language is not an issue, provided that components are compiled to the correct specification.

Distributed COM (DCOM) objects can communicate at a distance by marshalling on the client side and unmarshalling on the server side via proxies.

COM was not up to enterprise-level requirements – for example, it did not handle transactions well and had no support for security – so there was an extension called COM+, which received transaction and other enterprise-level support from another Microsoft component called the Microsoft Transaction Server.

Given the large number of these components in existence and the need for compatibility with legacy systems, .NET provides backward compatibility by wrapping communications with COM objects in XML.

6.2 Enterprise requirements

To help us to compare these platforms, let us review the motivations for enterprise systems, which have been discussed throughout this course.

There are functional and non-functional requirements for enterprise systems. In requirements engineering, non-functional requirements specify criteria that can be used to judge the operation of a system, and they are often referred to as the quality attributes of a system. In contrast, functional requirements specify the specific behaviour or function expected from the system – that is, something that the system is supposed to do or provide.

Non-functional requirements identified by Emmerich (2000) are:

- scalability;
- openness;
- heterogeneity;
- resource sharing;
- fault tolerance.

These requirements are hard to locate in any one part of the system and we will not attempt to discuss them here, except to say that both .NET and Java EE claim to support these requirements.

In addition, there are functional requirements such as:

- support for authentication of users;
- support for different levels of authorisation for code execution;
- support for transactions;
- access to utility classes and reusability of code;
- simple creation of dynamic web content;
- distributed communication;
- global namespaces;
- the ability to discover information about distributed services;
- off-the-shelf graphical components.

Any product claiming to solve enterprise system issues must address these and other requirements. Both the Java EE and .NET platforms aim to do so and take a very similar approach, with a few significant differences.

While Java EE promotes the use of a tiered architecture, the .NET framework is more service oriented, with messaging used for communication between components. However, both Java EE and .NET are systems comprised of a number of interacting parts that provide different services.

In this unit, we will not dwell on the architecture of these systems. You can imagine their components interacting in a hierarchical (layered) fashion, or you can think of them as simply being parts of a system or a pipeline. For either system, layers of communication can be skipped if direct communication with a part of the system seems desirable. Common middleware techniques such as marshalling and proxies also feature in both platforms.

We will also not attempt to address issues such as the speed of development on either platform, or the robustness and efficiency of applications in these environments.

6.3 The .NET approach

.NET takes a very similar approach to solving the issues of portability and security to that used in Java.

Both platforms provide runtime environments that manage a program's requirements from loading and linking through to removal from the system. In Java this is the Java Runtime; in .NET it is the **Common Language Runtime** (**CLR**). These components provide a layer of indirection above the operating system.

The CLR environment provides the same kinds of service as the Java Runtime, notably:

- a virtual machine;
- automatic memory management;
- garbage collection;
- array bounds checking to avoid buffer overflows (a security issue);
- utility classes (for example, to support database access).

The .NET virtual machine is built with native code execution in mind, so JIT (just-in-time) compilation or pre-compilation to native code is the norm.

.NET's environment also provides a kind of version control so that a client can determine which version of a server it might want to use.

Both platforms provide libraries of reusable classes, such as Java's core API. There are also APIs for specialised areas such as XML processing and SOAP messaging. Both provide extensive documentation of these libraries. Java also provides Javadoc, for building documentation for Java programs, while support for building documentation is more variable in languages supported by .NET.

Both platforms rely on intermediate languages: Java bytecode in Java's case, and the **Common Intermediate Language** (**CIL**) in the case of .NET, which is the compiled form of a number of implementation languages, including C#, VB.NET and J#, as illustrated in Figure 10.

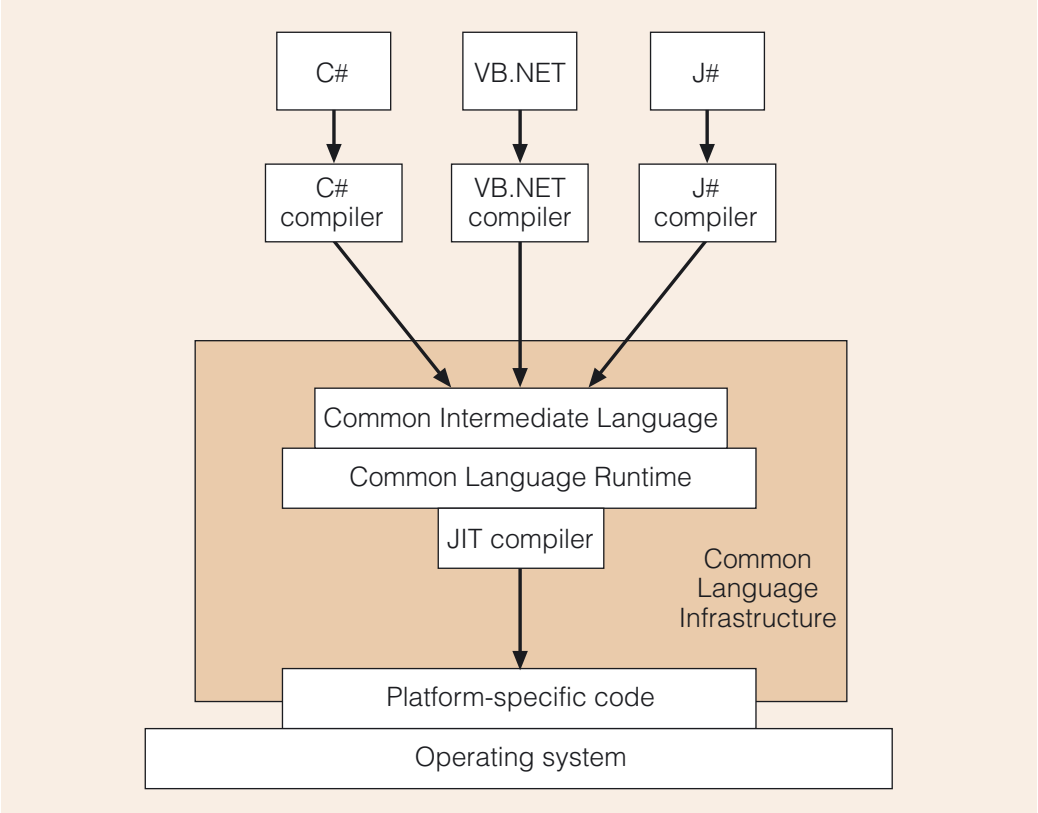


Figure 10 The Common Language Runtime infrastructure

Figure 10 shows that various implementation languages are compiled by their individual compilers into the common intermediate form known as CIL. A platform-specific CLR running on top of the operating system can then use a JIT compiler to mitigate the overheads associated with a virtual machine, and convert the intermediate language to platform-specific machine code.

The CLR is an implementation of part of the specification known as the **Common Language Infrastructure** (**CLI**), which can be implemented for different platforms.

The standard .NET platform is Microsoft Windows; however, support for earlier versions of Microsoft Windows has been dropped in later versions of .NET.

The .NET architecture relies on the same open standards as Java EE, in many cases including HTML, HTTP, XML and SOAP, and this in itself is a significant step towards interoperability.

Language issues

There are many commonalities between Java as a language and CLI-compatible languages. Naturally, both Java and .NET languages provide hierarchical namespaces, and an ability to import relevant classes from them. Both environments provide annotations (or metadata, as they are also known), which can be used by containers in deploying software components. Both environments support threading and reflection.

The most significant feature in this context is that in .NET various development languages may be used. This means that continued use of domain-specific languages, such as COBOL, which was designed with business programming in mind, is possible. Also, programmers will always have preferences for different languages, and the .NET framework gives a degree of flexibility in this respect.

As both C# and Java are descended from C and C++, they have many features in common.

Most commonly in .NET, development is undertaken in C# or in Visual Basic. If intermediate language compilers are created for other languages, they can be used to write software for the .NET framework.

As in CORBA, support for various languages requires mappings from an IDL to those languages as well as support for various types. This is the purpose of the **Microsoft Interface Definition Language** (**MIDL**), and also of the **Common Type System** (**CTS**), which is necessary to define mappings to and from types supported in various languages. (The latter serves the same purpose as the CDR in CORBA.)

As with other IDLs, supported languages may not map onto the intermediate language completely. Each must define a set of mappable language features, and possibly add some features (notably object-oriented features, in the case of older languages), in order to be translatable into the common intermediate language. For example, older programs in Visual Basic do not automatically run in the .NET framework because they do not provide object-oriented features. Thus VB.NET is a form of VB that provides object-oriented extensions.

Likewise, MIDL requires mappings to various languages supported by .NET such as C# and VB, while Java interfaces only map to Java.

Cross-compilers are available to produce Java bytecode from other implementation languages.

It is possible to obtain **cross-compilers** that will allow you to create compiled code for a platform other than the one on which the development is undertaken. So, in theory, Java EE also supports development in other languages for which cross-compilers to bytecode are available. For example, there are cross-compilers for versions of Python, Cobol and Eiffel. Similar limitations apply to these solutions – it may be possible to map only a subset of a language, and extra features may need to be added to the syntax of an existing language to take advantage of Java EE features.

6.4 Support for standard services

We have stressed in this course a variety of requirements for enterprise and distributed systems. Our aim in this subsection is to point out that both Java and .NET platforms provide libraries to handle common tasks in enterprise systems.

Both platforms provide messaging systems: the Java Messaging Service (JMS) in the case of Java, and Microsoft Message Queuing (MSMQ) in the case of .NET. In addition, built-in middleware can handle tasks such as load balancing for a server.

There is also high-level support for the configuration of services, such as, in the case of web services, setting the character encoding to be used, without the need to interact with low-level descriptions. Likewise both platforms allow configuration of security requirements for components within containers.

Feature comparison of .NET and Java

We now give a more complete breakdown of the key components in these enterprise development platforms.

In Table 2 we provide a comparison of the features in each platform. The names of the Microsoft components often reflect an older technology name. For example, dynamic web page construction has been provided for by Active Server Pages (ASP), which now appears as ASP.NET under the .NET umbrella. In some cases these technologies have undergone major transformations to fit in with the .NET platform.

Of course, it is difficult to compare a specification with a product. Our observation is merely that both platforms provide support for desirable features of enterprise systems. We do not expect you to be or to become conversant with all the acronyms in this table.

Table 2 Key features of .NET and Java EE

Feature	.NET	Java EE
Runtime environment	Common Language Runtime (CLR)	Java Runtime
Platform/portability	Wherever CLI is implemented, i.e. mainly recent Microsoft Windows and Unix-based platforms	Anywhere there is a Java Runtime and there are Java EE services (application server, etc.)
Dynamic web page construction[1]	ASP.NET (with various HTML output formats; a form of ASP) .NET web forms	JSP, servlet
Web services[2]	Windows Communication Foundation DISCO (a UDDI service) Related APIs	JAX-WS (web services) and related APIs, e.g. the JAXP API for XML parsing
Development languages[3] Both provide support for scripting languages	VB.NET, C#, Cobol, Smalltalk, Eiffel, Perl, J#, JScript, C++, APL; others possible	Java Other languages possible through cross-compilers (e.g. Python, COBOL, Eiffel)
Interface specification	MIDL OMG IDL (CORBA)	Java interface OMG IDL (CORBA)
JIT supported	Yes	Yes
GUI development	Windows forms, Windows Presentation Foundation	Swing, AWT and third-party GUIs such as SWT
Database access	ADO.NET (active data objects)	JDBC, JDO Java Persistence API

(continued overleaf)

Feature	.NET	Java EE
Remote object interaction, global namespaces	Yes	Yes
Compiles to	EXE, DLL containing CIL and metadata Assemblies and manifests	Bytecode with reflection built into objects to provide metadata JARs and manifests
IDEs	Visual Studio	Eclipse, NetBeans, JBuilder and many other free and proprietary IDEs

1 It is important to be able to deliver dynamic, personalised content to users. The most common way in which users will interact with systems is through browsers, so dynamic web page creation is an important feature in any system deployed on the internet. In .NET there is a feature called ASP.NET to create dynamic content.

2 Both have APIs supporting XML processing (DOM and SAX), public and private registry, service discovery and the generation of WSDL files.

3 For the .NET platform, language names refer to .NET versions of languages. For example, VB.NET is the .NET version of Visual Basic.

Interested readers may like to refer to the variety of reports available on the internet comparing the two systems; for example, discussion of the reference application called the Pet Store in .NET and Java.

Further reading.

What next?

Both platforms continue to evolve and are in widespread use.

Java Platform, Enterprise Edition 6 specification.

The release of Java EE 6 is expected in 2008. The specification states that 'The reach of the Java EE platform has become so broad that it has lost some of its original focus.' The new platform is to include 'profiles' targeted at particular application areas; for example, profiles including only a subset of relevant technologies. In addition, further support for web services is promised, as well as more support for service-oriented architecture, and updates to capabilities of various components, including EJB components and servlets.

While both .NET and Java EE are robust platforms for enterprise development, the learning curve for developers is significant in both cases. In some cases, a simpler approach may be preferred; for example, when all that is required is a web interface to a database.

JRuby is a version of Ruby that compiles to bytecode for the JVM while IronRuby is a version for .NET.

A recent contender for developing web applications is **Ruby on Rails**, which provides an object-oriented scripting language and simplifies the most common web deployment scenario, which is a web-based front end interacting with a database. Rails uses a model–view–controller (MVC) pattern, the Ruby programming language, and **scaffolding** to enable rapid development. A scaffold uses a database access specification to provide a starting point for development and this approach means that many commonly required database access tasks need not be explicitly coded. Instead, a developer can concentrate on developing the exceptions to the usual cases.

In the context of simple applications involving a web interface to the database, the thorough support provided by Java EE and .NET may be considered unnecessary and the rapid development possible with Ruby on Rails an advantage.

Interested readers may refer to many recent titles on the subject; for example, a high-level discussion is found in Tate (2005).

Exercise 4

What are the key differences and similarities between .NET and Java EE?

Discussion .....

Similarities include:

- Application to enterprise systems and support for enterprise requirements such as transaction processing, web services, security, SOAP/XML and CORBA.
- Both rely on an intermediate layer above the OS: the Java Runtime and the CLR for .NET. This has speed implications for both platforms. Both rely on a virtual machine.
- Both rely on an intermediate language: bytecode is executed by the JVM within the Java Runtime, and the CLI in the case of .NET.

Differences include:

- Homogeneous application language in Java EE (unless external support is used, such as CORBA) versus heterogeneous implementation language in .NET through the CIL. This may make .NET easier to use to integrate with legacy systems.
- Java EE is a specification with many implementations, whereas .NET is a Microsoft framework primarily for Microsoft Windows platforms. There are various IDEs for Java, while .NET development is limited mainly to Microsoft Visual Studio.
- There is an open-source version of Java EE, while .NET applies a stricter licence. Vendor independence is seen as an advantage of Java EE.

7 Summary

In this unit we have discussed some general approaches to achieving heterogeneous communication in distributed systems. Layers of indirection are a common solution to providing interoperability between components that operate with different languages. Likewise, components and interfaces are important in providing interoperability and reusability of software.

We looked at an object communication system in the form of CORBA, a specification produced by the OMG. CORBA has many features in common with RMI, including the use of proxies and marshalling for communication. In CORBA, an ORB acts as a middleware between a client and a server to achieve distributed object communication. In addition, CORBA specifies a number of standard distributed system- and domain-specific services.

XML presents a generic way of describing messages in the context of heterogeneous systems, while SOAP provides a lightweight means of sending such messages, typically over HTTP. Such common languages are useful in facilitating communication between components deployed on different platforms and in different languages.

XML, SOAP and HTTP also form the most common basis of web services. A web service is an example of the service-oriented paradigm in which a client can access a server via a browser or dedicated client. A client can use a service discovery mechanism to determine what services are available, and can find out how to use such a service if it is described using WSDL, an example of an interface language.

Both Java EE and .NET provide platforms supporting the development of enterprise software, including support for web service development, GUI development, dynamic web pages, database access and other standard enterprise services such as transactions and security. These platforms can also interact with each other and with CORBA applications.

The computing landscape is continually changing, however, with new technologies emerging and older ones losing favour or finding their niches.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- explain what middleware is, and explain the roles of layers of indirection and components in heterogeneous communication;
- describe the use of an interface definition language (IDL) and name examples of such languages;
- describe the role of the Object Management Group (OMG) and the characteristics of the OMA Reference Model, including CORBA (the Common Object Request Broker Architecture), and the role of an ORB;
- describe the format of an XML message and why it is useful for message passing;

- describe the format of a SOAP message and discuss the use of SOAP in web services;
- describe the operation of web services and be able to implement simple web services using NetBeans;
- suggest some scenarios in which CORBA, RMI, SOAP or web services might be preferred, and highlight key similarities and differences between them;
- outline key similarities and differences between the Java EE and .NET environments.

Glossary

application object A user-defined object in a CORBA application.

at-most-once A semantics of invocation in which the invocation will block and will either successfully receive a result or fail due to an exception, resulting in a synchronous communication.

attribute (XML) An attribute describes the properties of an element.

client (CORBA) A CORBA object acting in the role of a client in some operation.

Common Data Representation (CDR) A set of data types that may be used to represent types in various implementation languages in CORBA interfaces.

Common Intermediate Language (CIL) In .NET, the common intermediate language to which all languages are compiled – equivalent to bytecode in Java.

Common Language Infrastructure (CLI) In .NET, the collection of technologies allowing various implementation languages to run on different platforms.

Common Language Runtime (CLR) The .NET environment that performs the task of just-in-time compilation of CIL to a platform-dependent code, and is also responsible for the lifecycle of .NET processes, including garbage collection.

Common Object Request Broker Architecture (CORBA) The part of the OMG specification commonly known as CORBA, which specifies the properties an ORB must have to be CORBA compliant.

common service In CORBA, common services are generic services that are useful across applications and domains, including support for printing, mobile agents and internationalisation.

Common Type System (CTS) In .NET, the CTS defines mappings to and from supported implementation languages and the Microsoft Intermediate Language (MSIL).

component model A set of standards describing the characteristics of a type of component, governing the implementation, documentation and deployment of components.

container (web service) An environment used to deploy a Java EE or CORBA component and providing services to that component such as support for security and transactions.

CORBA compliant In CORBA, software implementing the CORBA specification is said to be CORBA compliant.

CORBA services Services that are often needed in distributed applications, such as support for naming, transactions and security. See **service**.

cross-compiler A compiler that will allow the creation of compiled code for a platform other than the one on which the development is undertaken.

discovery service A component whose job it is to store details about services that are available on a network and respond to client requests for information about those services.

Document Object Model (DOM) A platform-independent XML validation standard defining how to read and manipulate an HTML or XML document as an object representing a tree data structure, and providing random access to XML elements.

document type definition (DTD) A DTD describes valid element types for a particular type of XML document (having a particular root element) and the relationships of the elements to each other.

domain service CORBA services that are needed within particular application domains, such as medical or financial systems.

Dynamic Invocation Interface In CORBA, this interface allows a client to bypass a stub and interact directly with an ORB's services.

Dynamic Skeleton Interface In CORBA, this interface allows a server to dynamically create a new interface.

element (XML) A semantic markup of text, approximately equivalent to a type.

eXtensible Markup Language (XML) A form of semantic markup of text similar to HTML.

heterogeneous system A system implemented using more than one kind of computer, operating system or communication protocol and particularly using more than one programming language.

IDL compiler A compiler for an interface definition language (IDL).

implementation repository In CORBA, a database used by a server to register information about servant objects and enabling a client to access servant operations.

indirection The general technique of using an intermediate layer to facilitate communication between two parts of a system.

interface definition language (IDL) An IDL describes the message-passing ability of components in terms of operation names and parameters and provides a layer of indirection above the component.

interface repository A database of interface information in CORBA, similar to the registry in Remote Method Invocation (RMI).

interface (CORBA) The general term for descriptions of components' message-passing ability in terms of their operation names and parameters.

Internet Inter-ORB Protocol (IIOP) A protocol for use in inter-ORB communication in CORBA applications, and providing support for distributed communication.

interoperability The property of being usable by other systems, particularly in a heterogeneous system context.

lookup service The part of a discovery service used by clients to ask for information about registered services. The discovery service can then enable clients to choose between various services, or it can choose a service for them.

loose coupling A loosely coupled component is insulated from changes in another component due to the presence of a layer of indirection between them.

mapping Rules for translating from a language to an interface definition language, including specification of the relationships between types in communicating idioms.

Microsoft Interface Definition Language (MIDL) The language used to describe the interface to a .NET object.

middleware Any software acting as an intermediary between two components, and particularly software that hides the heterogeneity of components, thus facilitating interoperability.

mode A modifier for a parameter in an operation definition, such as in CORBA's interfaces, describing what may legally be done with the parameter. For example, whether the parameter may be read from, or written to.

namespace A hierarchical partitioning of names that provides a way of uniquely identifying any name in a context such as an XML document or software application.

namespace prefix The use of a namespace name as a qualifier for the simple name of an element or attribute, so forming an extended name.

object (CORBA) In CORBA, the general term for an object within a CORBA application, incorporating user-defined client and server objects as well as those implementing standard specifications such as those contained in common services.

object adaptor A component facilitating communication between an ORB and a CORBA servant.

Object Management Architecture (OMA) The architecture produced by the OMG, of which CORBA is a part; described as an 'open, vendor independent specification for an architecture and infrastructure that computer applications use to work together over networks'.

Object Management Group (OMG) The group responsible for the Object Management Architecture (OMA).

object service (CORBA) A general description for operations provided by objects in the context of CORBA.

object type (CORBA) In CORBA, a type defined by an interface within a module, specifying message-passing behaviour for a class of objects in a CORBA application.

OMG Interface Definition Language (OMG IDL) The interface definition language specified by the Object Management Group for use in CORBA. See **interface definition language (IDL)**.

OMG Object Model A platform-independent description of CORBA objects, operations, interfaces and types.

operation (CORBA) A general term for the message-passing ability of components in CORBA applications, roughly equivalent to a method.

prologue The first part of an XML document, which declares its version number and may include information such as the encoding (character set).

qualified name A name formed from a prefix and a simple name in XML.

registration service A component providing a way of registering interfaces in a repository to allow a client to perform service discovery.

resolving a name The process by which an object reference is associated with a particular servant object in a CORBA application; a binding of a reference to an object name.

root element The first element in an XML document, which encloses other elements and acts as a way of identifying the type of a document.

Ruby on Rails An approach to the development of web-based access to databases that uses a database specification, and scaffolding to provide the most commonly required operations.

scaffolding The technique of providing an infrastructure delivering commonly required operations and allowing modification for special cases, so facilitating rapid development.

scope of discovery The extent to which a service discovery application will search for a service matching a request.

servant An object, under the control of a server, providing a CORBA operation to a client.

server (CORBA) A CORBA object acting in the role of a server in some operation.

service A generic term describing the message-passing ability of components, but particularly used in service-oriented systems.

service-oriented architecture An architecture in which the main unit of communication is a message as opposed to a method invocation. Service providers use an interface to describe their services, and can be connected together in arbitrary configurations.

skeleton A layer between a servant and an ORB in a CORBA application.

Simple API for XML (SAX) An API providing access to XML as a stream of data and allowing serial access to elements, but not random access.

simple name A name used to uniquely identify an XML document's namespace.

SOAP An XML-based protocol supporting communication between distributed systems and widely used in web services.

software component A software element that conforms to certain requirements for interoperability defined by a component object model. In particular, software components support the goal of composability to create higher functionality than they individually provide.

stub A layer between a client and an ORB in a CORBA application.

tight coupling A tightly coupled component communicates directly with another in such a way that changes in one are liable to affect the other.

Universal Description, Discovery and Integration (UDDI) An XML-based standard used by a server to register WSDL descriptions of web services and by a client to discover such services. As such it allows the creation of directories for web services.

valid An XML document is valid if it is well formed and conforms to a DTD.

web service An example of the network services paradigm in which software services are provided via the Web, particularly using HTTP, web pages and SOAP as communication mechanisms.

Web Services Description Language (WSDL) A language used by UDDI for the description of web service operations using XML.

well formed The property of an XML document of being syntactically (as opposed to semantically) correct.

XML See **eXtensible Markup Language (XML)**.

XML Schema Definition (XSD) An XML-based description of an XML document, allowing for better type checking than a DTD and more rigorous constraints for XML documents to meet.

References

Clark, B. (2005) *Enterprise Application Integration Using .NET (The Addison-Wesley Microsoft Technology Series)*, Addison-Wesley.

Emmerich, W. (2000) *Engineering Distributed Objects*, Wiley.

LG Electronics (2002) *Digital Multimedia Side-by-side Fridge Freezer with LCD Display* [online], http://www.lginternetfamily.com/fridge.asp (Accessed 27 March 2008).

Object Management Group (2007) *Introduction to OMG's Specifications* [online], http://www.omg.org/gettingstarted/specintro.htm (Accessed 3 January 2008).

Platt, D.S. (2004) *The Platform Ahead*, Microsoft Press.

Sun Microsystems (2002) *Java RMI over IIOP* [online], http://java.sun.com/j2se/1.4.2/docs/guide/rmi-iiop/ (Accessed 3 January 2008).

Tate, B. (2005) *Beyond Java*, O'Reilly.

Index